

第11 分布式系统 架构—微服务

北京交通大学软件学院

严禁复制



目录



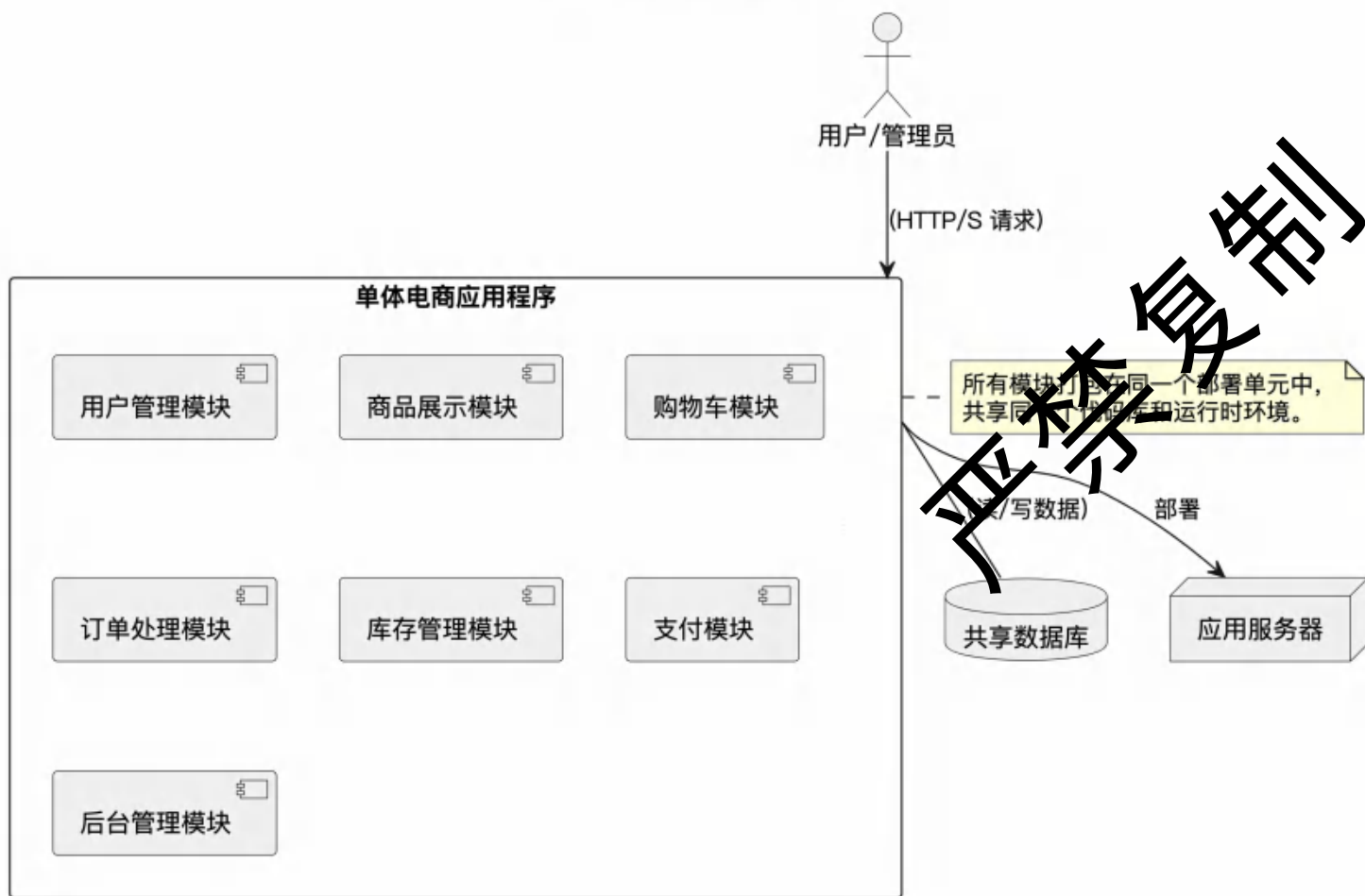
01. 分布式系统的区别
02. 微服务模式
03. 微服务的挑战
04. 微服务应用模式



01 复制 分布式系统的区别

单体架构

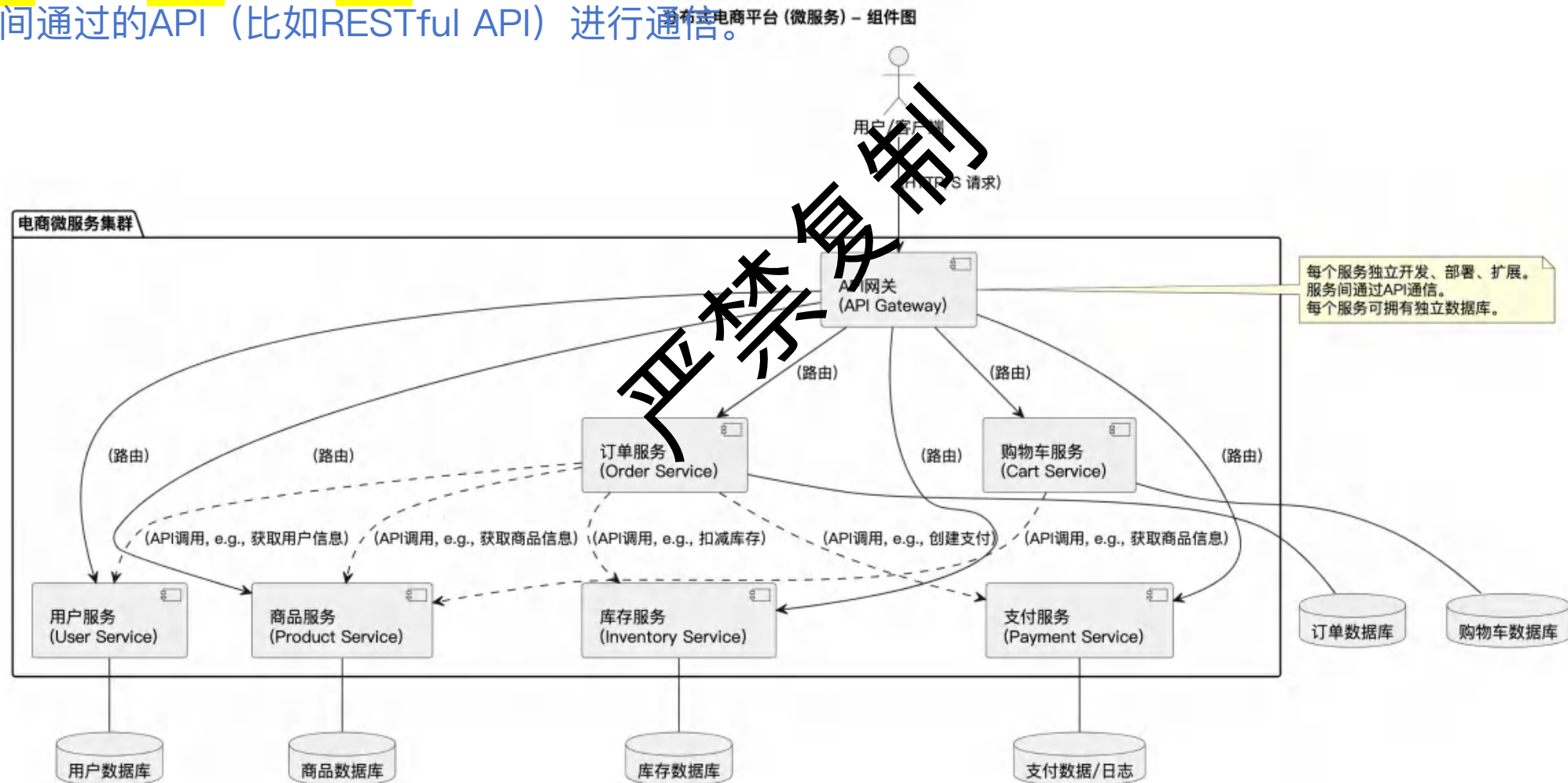
单体电商平台 - 组件图



- 定义:
 - 所有这些模块的代码都存在于同一个代码库中。
 - 最终会被编译、打包并部署到一台或多台服务器上。
 - 即使部署到多台服务器, 运行的也是**同一个**完整的应用程序实例。
- 优点:
 - 开发初期简单直接,
 - 本地函数调用, 效率高,
 - 部署也相对容易。
- 缺点:
 - 扩展性差。
 - 技术栈单一:
 - 可靠性低:
 - 维护困难:
 - 部署不灵活:

分布式架构（以微服务为例）

每个服务都有自己独立的代码仓库和数据库（或者共享数据库但逻辑隔离），可以独立开发、独立测试、独立部署和独立扩展。它们之间通过的API（比如RESTful API）进行通信。



单体架构VS分布式架构

特性	单体架构	分布式架构
部署单元	单个、庞大的应用程序	多个、小型的独立服务
扩展性	整体扩展，不够灵活	按需独立扩展服务，更灵活高效
技术栈	通常统一	可异构，不同服务可用不同技术
可靠性	单点故障风险高	更好的故障隔离和容错性
开发维护	随规模增大而急剧复杂化	单个服务相对简单，但整体协调复杂
团队协作	大型团队协作困难	更适合多团队并行开发
通信方式	进程内函数调用	跨进程/跨网络API调用（如HTTP, RPC）

分布式架构模式概述

- 系统集成模式：
 - 文件传输，共享数据库，远程调用，消息传递
- 拓扑结构模式：
 - 中心辐射型 (Hub & Spoke): 所有通信都通过一个中心节点。
 - 总线型 (Bus): 所有组件都连接到一个共享的通信总线。
- 代理模式：
 - 中间代理来解耦消息的发送者和接收者。
- 点对点模式：
 - 既是客户端也是服务器，直接相互通信。
- 面向服务架构 (SOA):
 - WS-* (Web Services Standards): 基于XML的重量级标准。
 - REST (Representational State Transfer): 一种更轻量级的、基于HTTP的架构风格。
- 微服务架构 (Microservices):
- 无服务器架构 (Serverless):

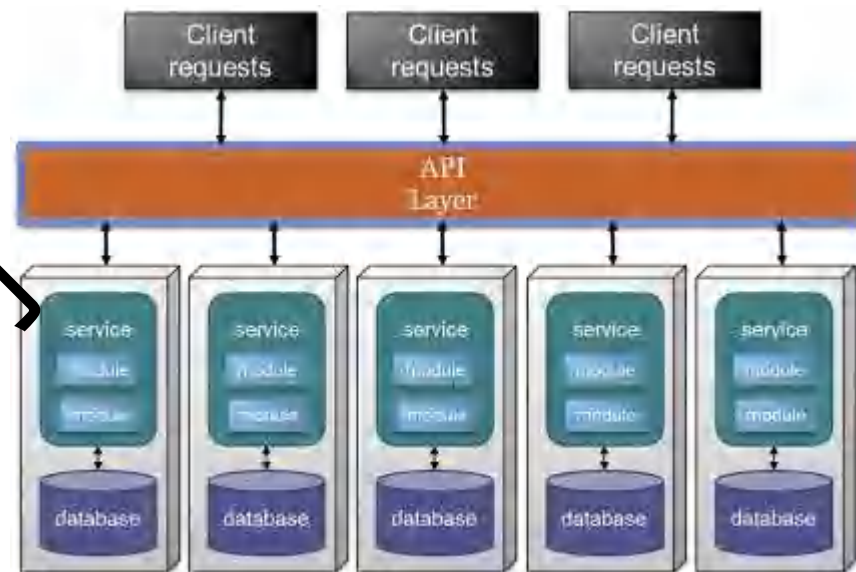
严禁复制



02 复制微服务模式

微服务架构

- 提出时间2014年，核心原则
 - 单一职责
 - 独立部署
 - 轻量级通讯
 - 去中心化治理
- 技术支持
 - Docker：打包和部署
 - K8s：集群管理
- 对比
 - 单体模型
 - SOA



<https://martinfowler.com/articles/microservices.html>

微服务的约束

- 分布式部署
 - 不要把多个服务部署在一台服务器上
- 限界上下文
 - 每个服务为一个领域建模，内部保持语义无歧义
- 数据隔离
 - 互相不能直接读写db
- 互相独立
 - 没有中间人和协调者

严禁复制

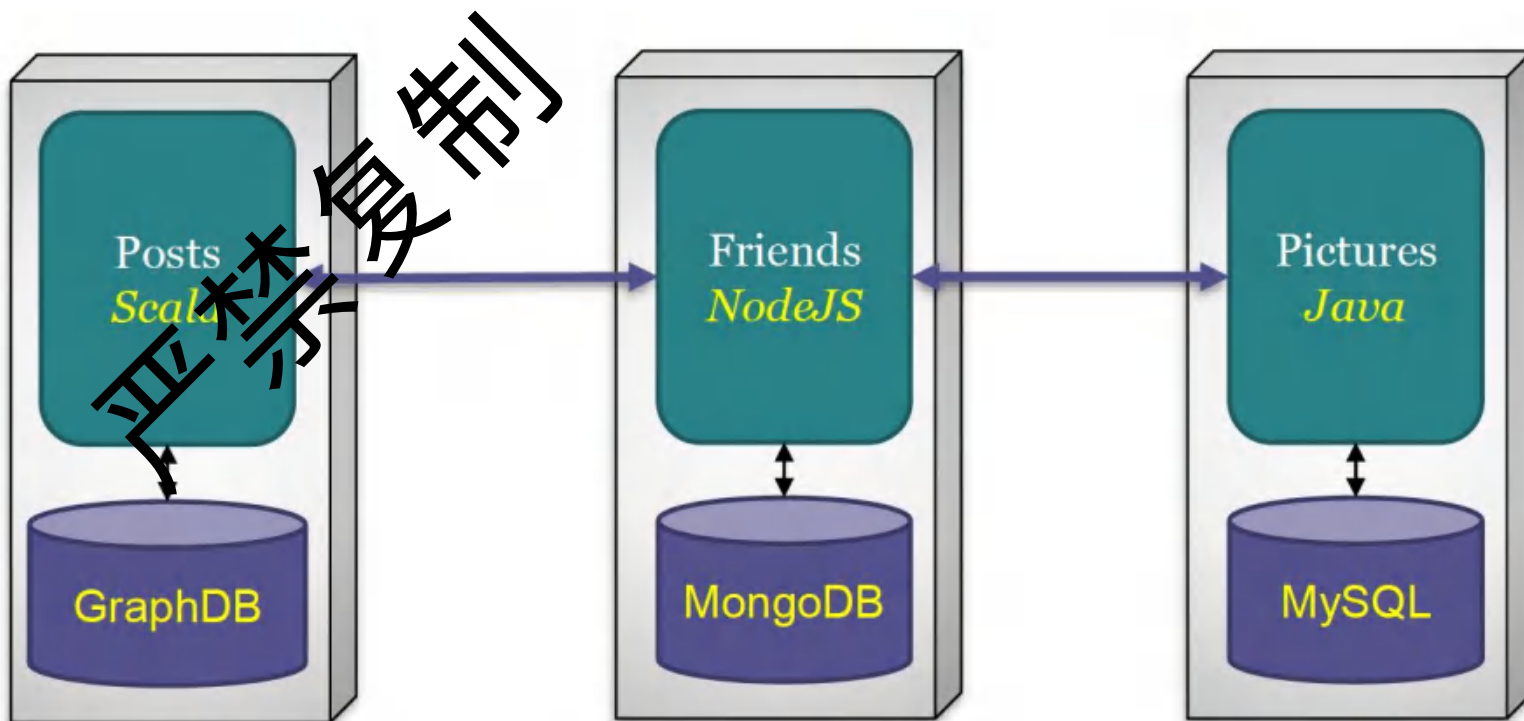
微服务的优势

1. 易于团队分工协作
2. 支持多种的技术栈
3. 容易恢复
4. 伸缩性好
5. 部署成本低
6. 避免中心数据库过于复杂
7. 解耦合，可替换，支持架构进化

严禁复制

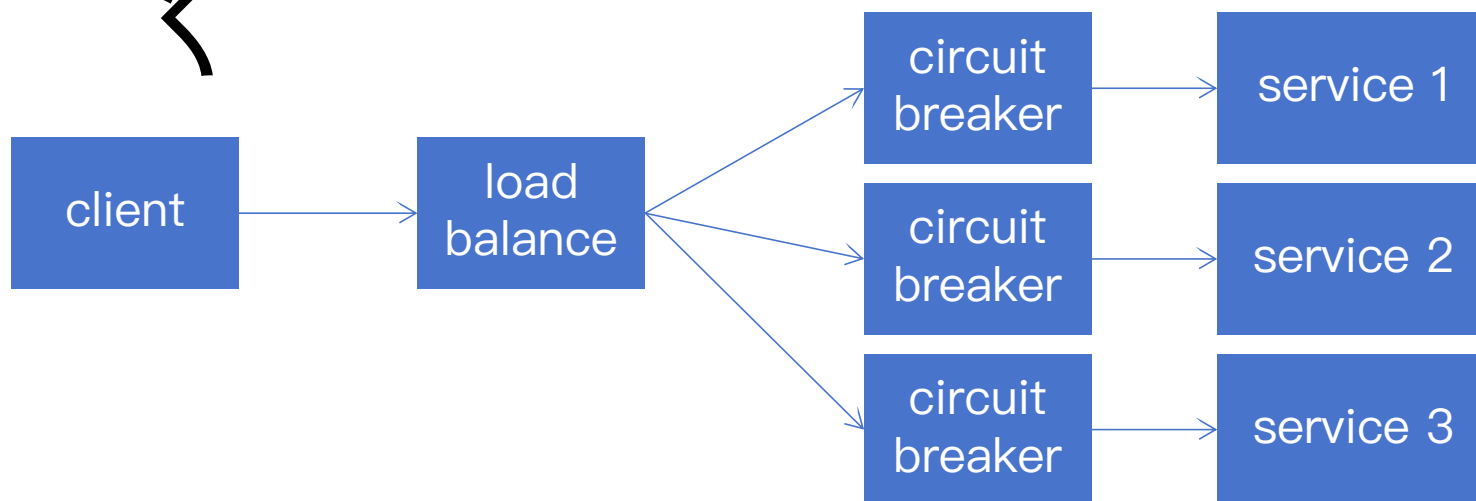
1 技术异构

- 每个微服务可以使用自己的language和technology stack
 - 可以试验新技术
 - 灵活选择



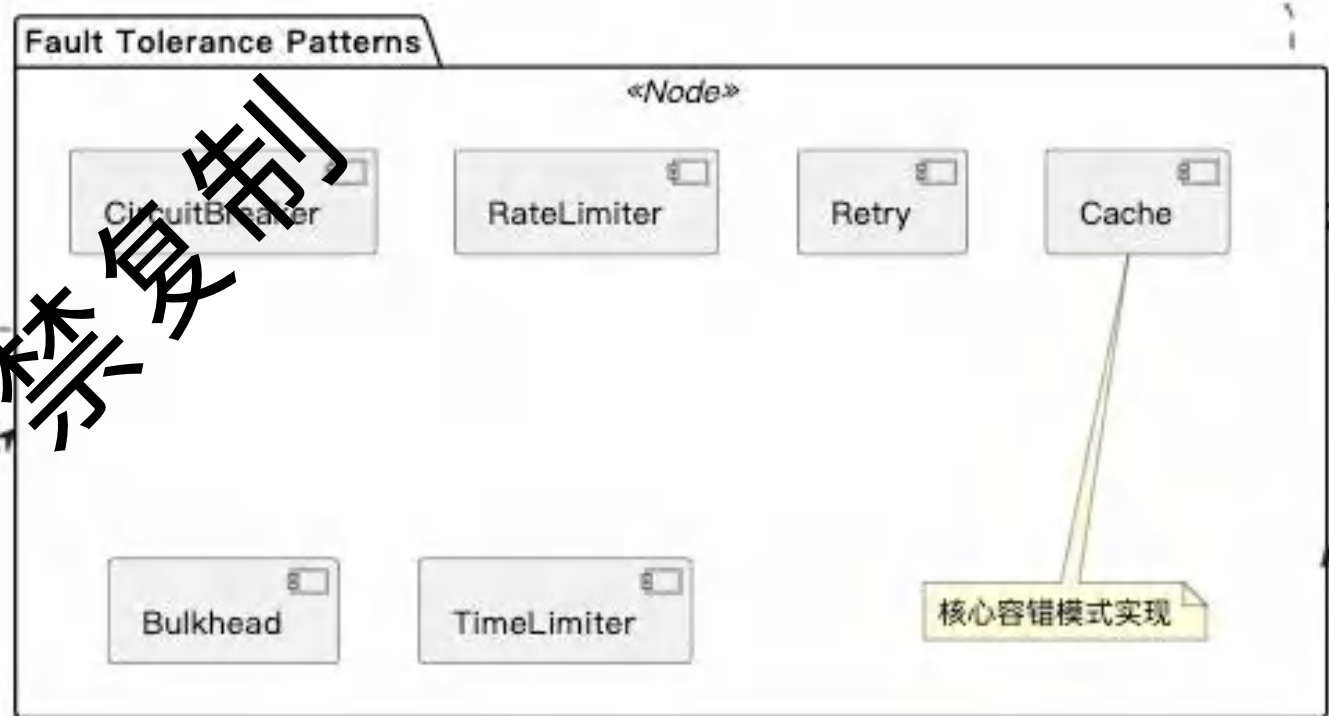
2 容错恢复(断路器模式)

- 微服务系统 VS 单体系统：
 - 每个微服务独立部署，一个服务failed了，不影响整个系统
 - 所有模块部署在一起，一个组件failed了整个系统stop working
- 断路器模式 (Circuit Breaker)
 - 监控调用：调用成功，失败，超时次数
 - 状态转换：closed, half-open, open
 - 降级处理：备选服务
- 断路器实现
 - Hystrix
 - resilience4j
 - Sentinel



2-a Resilience4j 核心模块

- resilience4j-circuitbreaker:
 - 失败率超过阈值时阻止，后尝试恢复。
- resilience4j-ratelimiter:
 - 限流器模式，防止系统过载。
- resilience4j-retry:
 - 实现重试机制。可配置重试次数、等待间隔、针对特定异常重试等。
- resilience4j-bulkhead:
 - 实现舱壁隔离模式。限制并发执行的许可数量。
- resilience4j-timelimiter:
 - 实现超时限制。
- resilience4j-cache:
 - 提供缓存支持，缓存方法调用的结果。



2-b Resilience4j 使用例子

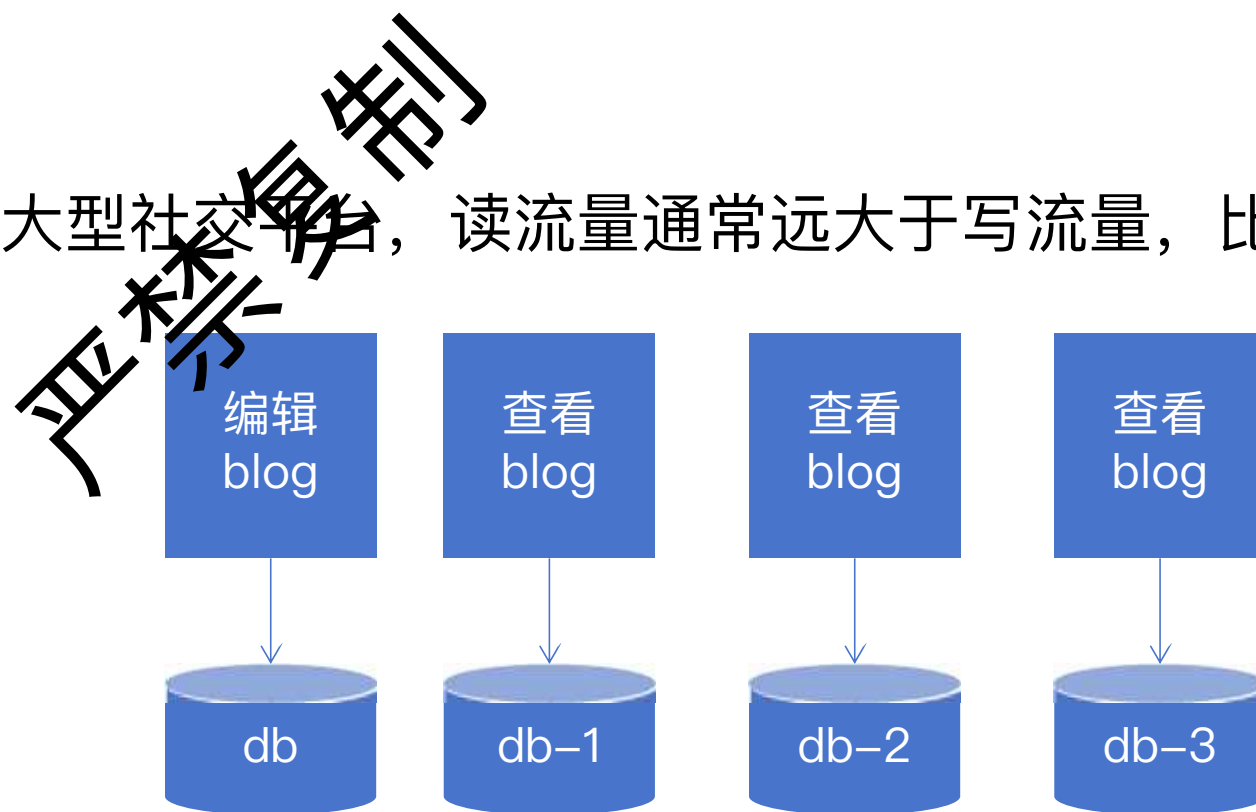
对supplier服务进行装饰，提升它的容错能力。

```
CompletableFuture<String> future = Decorators.ofSupplier(supplier)
    .withThreadPoolBulkhead(threadPoolBulkhead)
    .withTimeLimiter(timeLimiter, scheduledExecutorService)
    .withCircuitBreaker(circuitBreaker)
    .withFallback(asList(TimeoutException.class, CallNotPermittedException.class),
        throwable -> "Hello from Recovery")
    .get().toCompletableFuture();
```

<https://github.com/Imhmhl/Resilience4j-Guides-Chinese/blob/main/examples/circuitbreaker-example.md>

3 伸缩性

- 可以按需扩展特定的服务。
 - 单体系统需要扩展整个系统。
 - 并非所有组件都有相同的需求。
 - 微服务可以按需进行复制。
- 例：在X.com（Twitter）这类大型社交平台，读流量通常远大于写流量，比例一般在100:1甚至更高。
 - 写blog
 - 查看blog



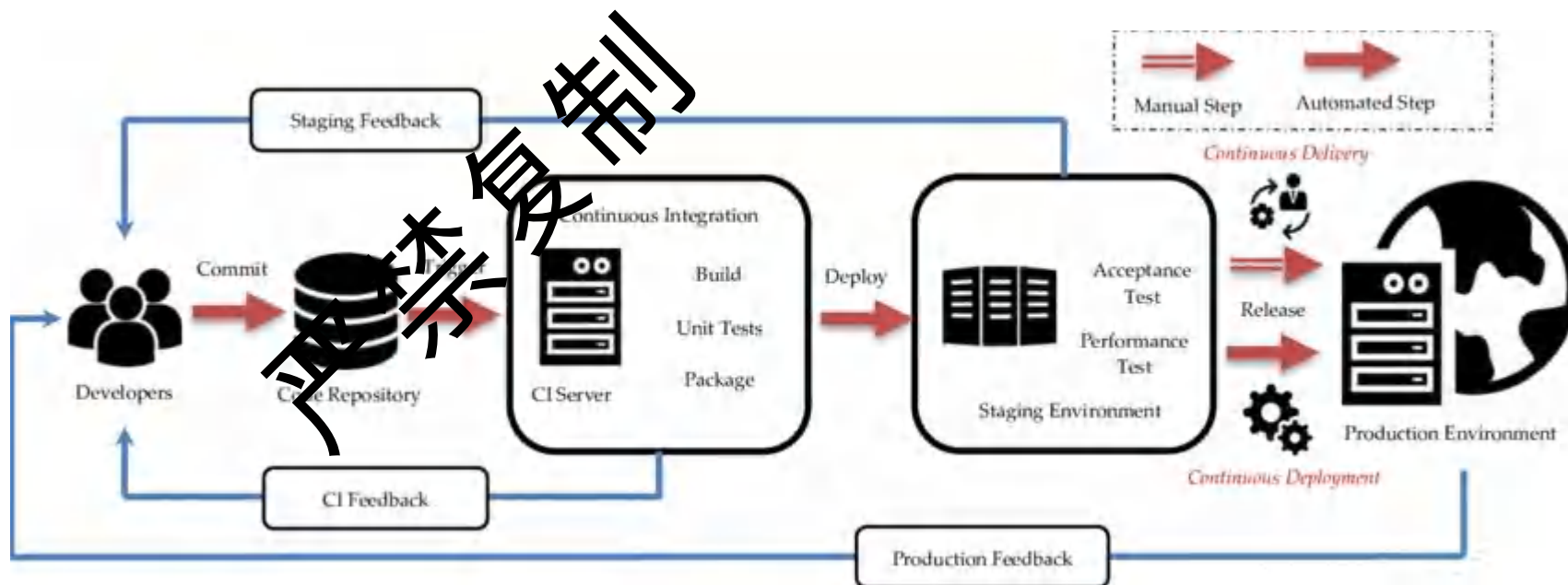
4 易于部署

- 特点

- 快速完成发布流程
- 持续集成，持续部署

- 对比单体架构：

- 第三方依赖更少
- 机器资源更少
 - cpu, 内存
- 开发人数更少
 - 容易协调



5 有利于团队分工协作（明确责任）

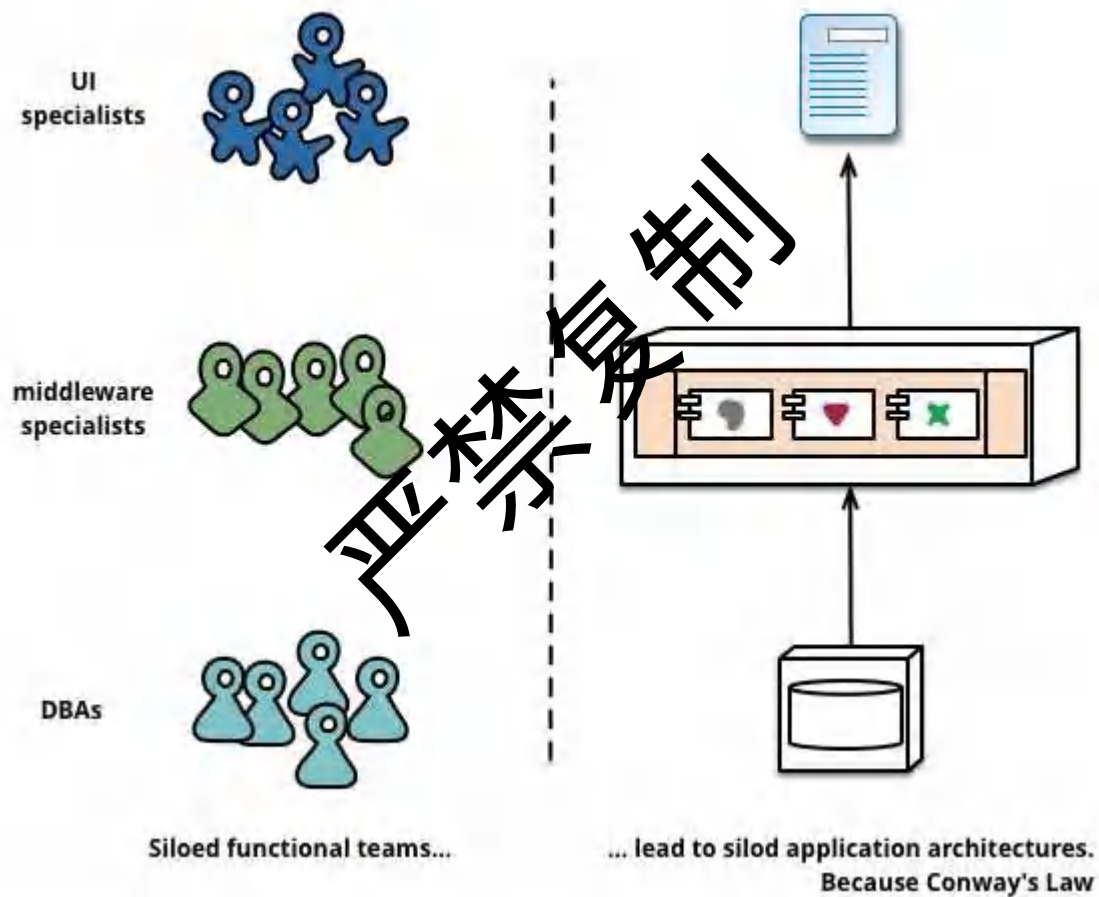
- 逆向康威定律的运用
 - 演进团队和组织结构，以促进期望的架构
- 按照模块化分解创建团队
 - 跨职能团队（前端，后端，DB，运维）
- 服务所有权：拥有服务的团队负责对其进行变更和部署
 - “谁构建，谁运维”（亚马逊）
- 目标：提高自主性和交付速度

5-a 传统研发团队（职责不清）

只为前端负责

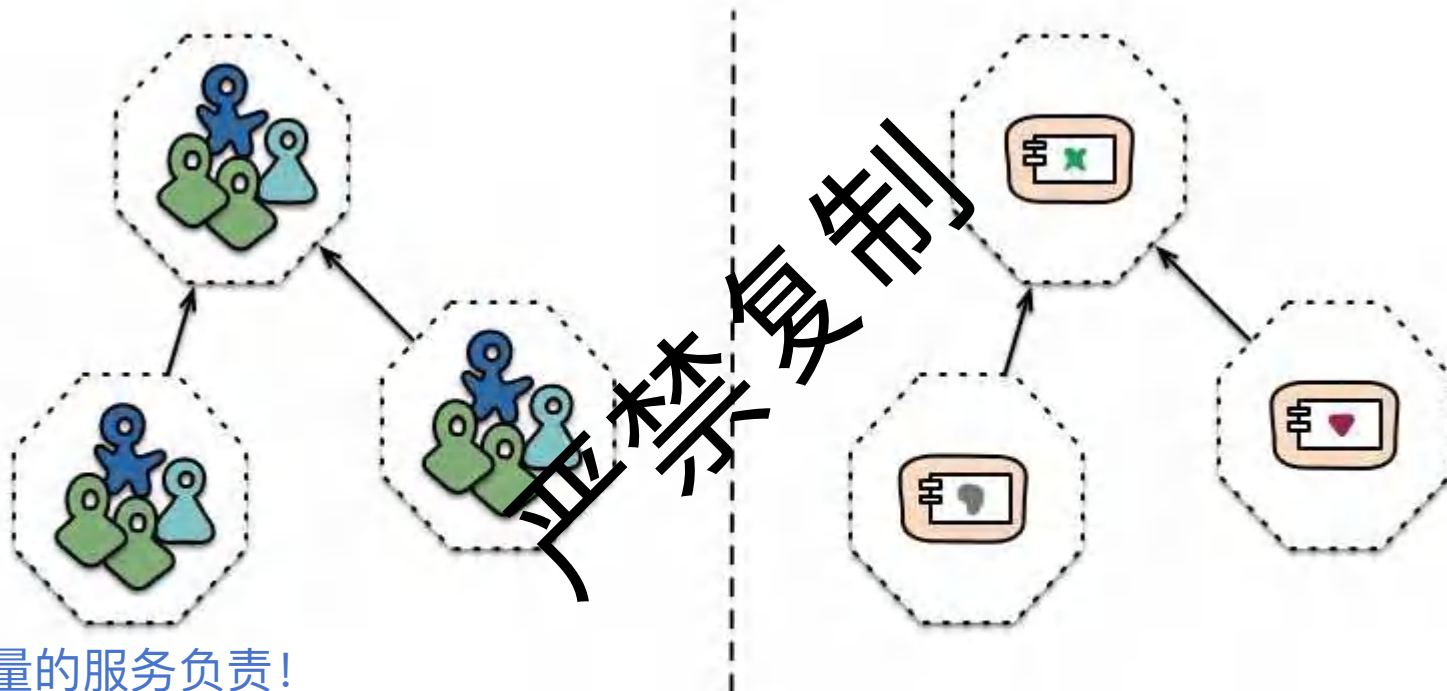
只为后端负责

只为DB负责



谁为系统整体负责？

5-b 微服务研发团队（职责清晰）



一起为高质量的服务负责！

Cross-functional teams...

... organised around capabilities
Because Conway's Law

6 微服务数据库的优势

- 服务解耦合：独立数据库，减少耦合。
- 灵活选型：数据库类型（如关系型、NoSQL、时序数据库等）。
- 系统可扩展：热点服务单独扩展，提高整个系统的可伸缩性。
- 提升稳定性：单个服务故障时，不会影响到整个系统。
- 数据安全管控：根据数据敏感性，实现更精细的数据管理。
- 简化数据库迁移：数据库迁移和升级风险降低。
- 减少全局锁竞争：降低了资源争用和全局锁的发生概率，提高了性能。

7 微服务具备可替代性

- 单体架构：
 - 历史遗留系统无人运维，不敢下线。
- 微服务架构：
 - 替换单个微服务，低成本，保持稳定
 - 甚至可以直接删除某个微服务
- 进化式架构
 - 可持续演进：逐步适应新的业务。
 - 可替换性和灵活性：替换、升级或添加新功能。
 - 自动化和反馈机制：自动化测试、持续集成和持续交付，及时获得反馈。
 - 架构决策的可逆性：避免“不可逆”的技术选型，可以试错和回退。

严禁复制



03 复制 微服务的挑战

微服务的挑战

- 管理的挑战
 - 需要管理的微服务数量很多，微服务过多 = 反模式（纳米服务）
- 分布式系统的特性
 - 新的挑战：数据一致性，延迟、消息格式、负载均衡、容错等
- 测试和部署的复杂性
 - 端到端的测试变得困难
 - 服务之间并没有完全独立
- 架构演进中的风险
 - 反模式：分布式单体
 - 结构性腐化（详见下一页）系统结构随时间推移逐渐变得混乱

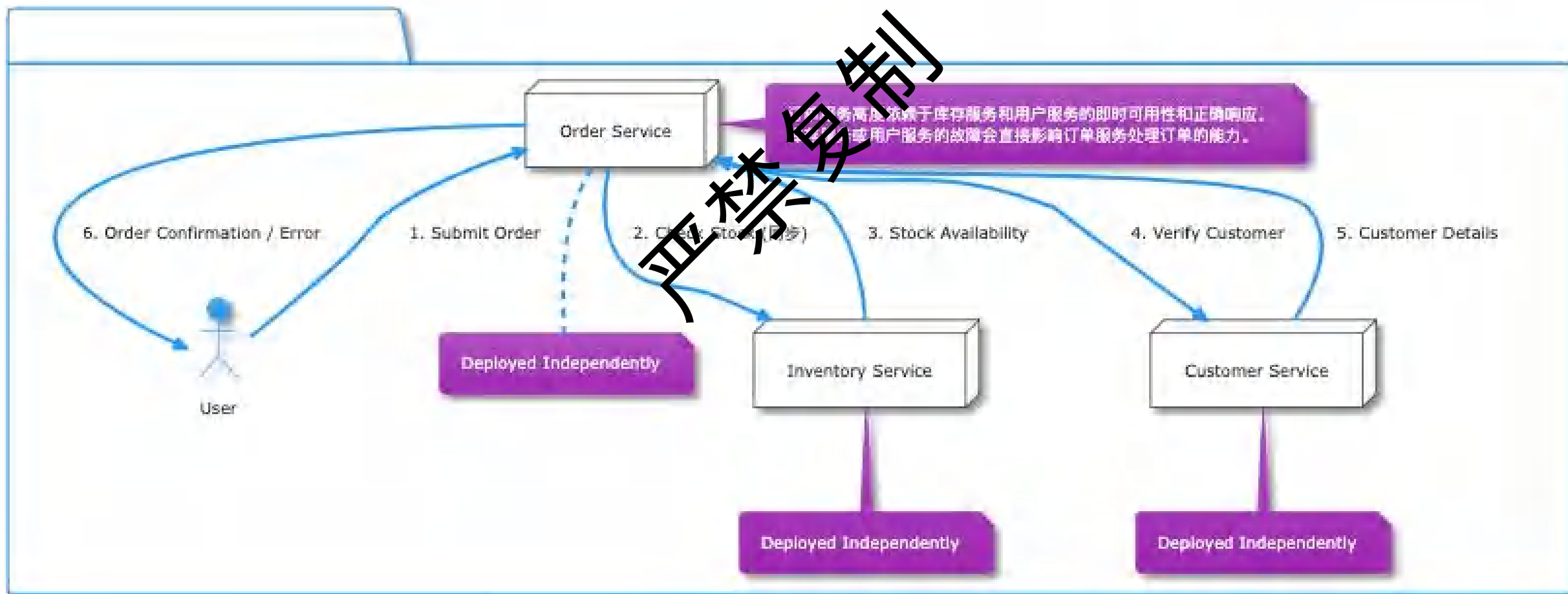
禁止复制

微服务挑战：伪分布式（分布式单体）

问题：订单服务高度依赖于库存服务和用户服务的即时可用性和正确响应。

库存服务或用户服务的故障会直接影响订单服务处理订单的能力。

解决方案：order service，仅在必要时才调用，保持Order Service的完整性

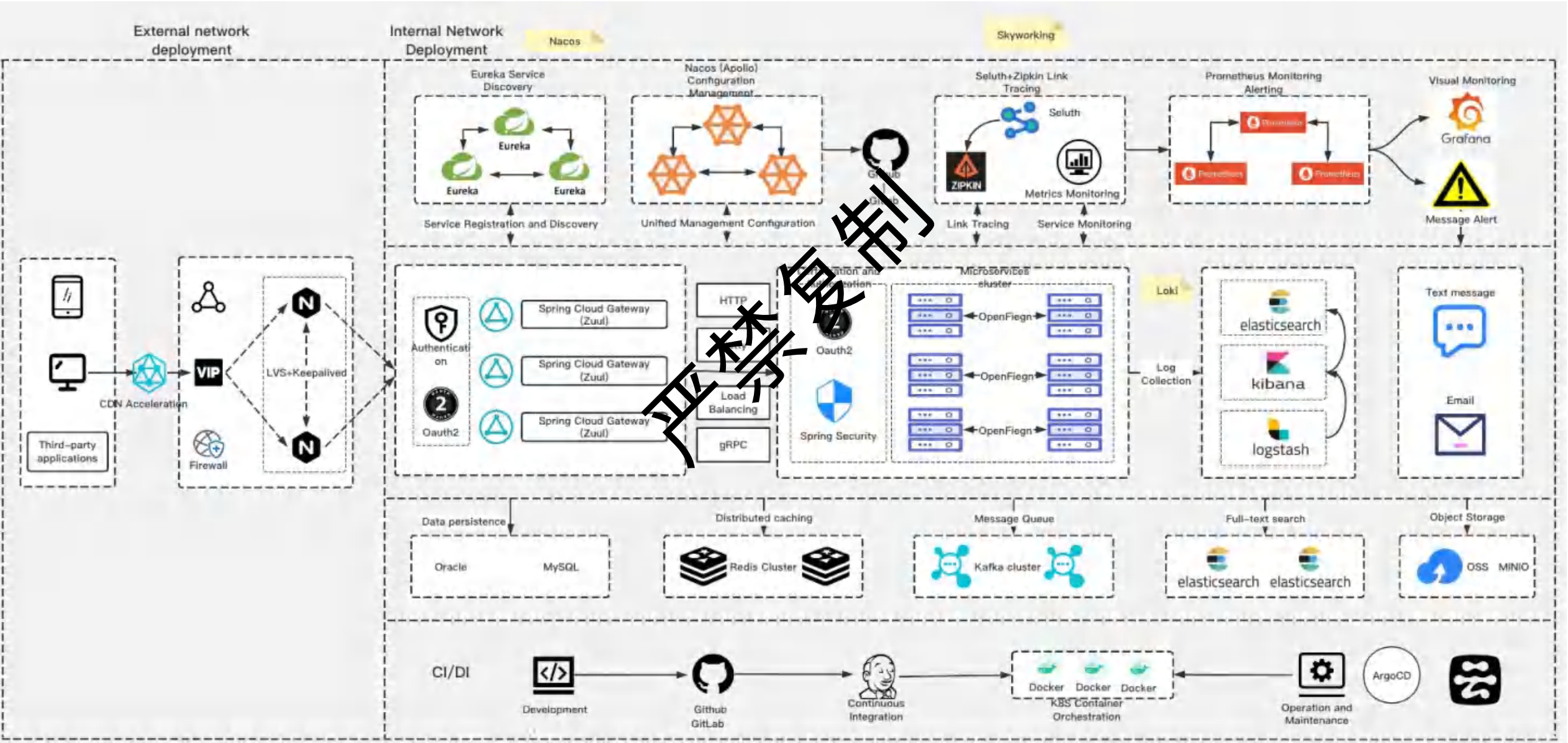


微服务挑战：结构性腐化

- 服务之间的代码依赖
 - 多个微服务共同依赖一个jar包，修改后必须同时升级
- 服务间通信过于频繁
 - 服务之间互相调用，不能独立完成任务
- 编排请求过多
 - 说明微服务粒度太细，需要组合才能完成用户需求
- 数据库耦合
 - 需要把某个数据从一个db导入到另一个db

禁止转载

微服务的运行环境



运行环境中的关键系统

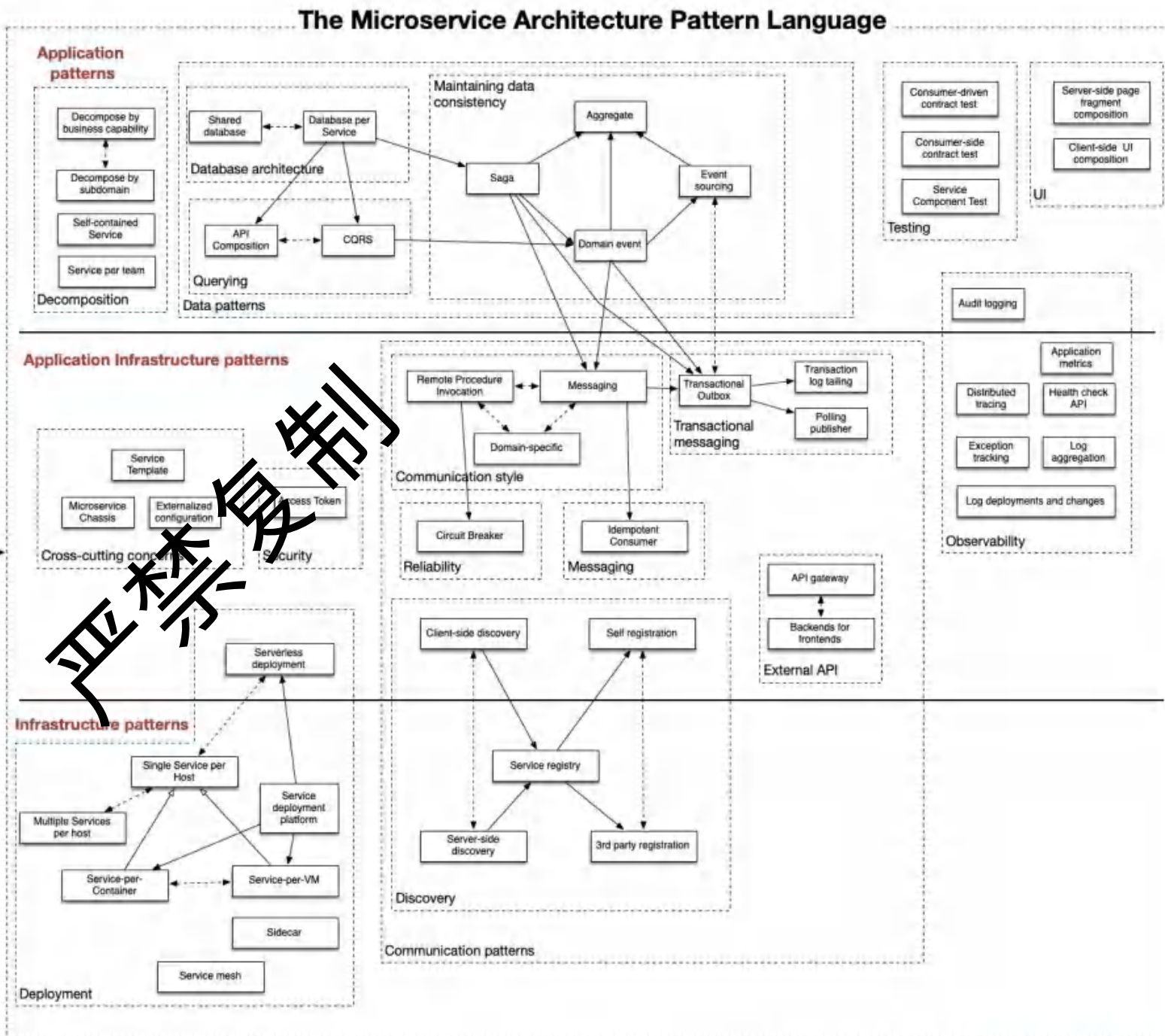
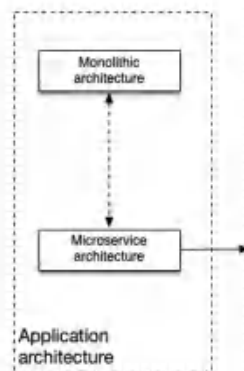
- Eureka/Nacos 帮你找到其他服务。Nacos 还能帮你管理配置。
- Zuul 是所有请求的入口大门，负责路由和一些通用处理。
- Sleuth + Zipkin 帮你追踪一个请求在各个服务间的完整旅程，方便排查问题。
- Prometheus 负责收集各种系统和应用的运行指标，并在出问题时告警。
- Grafana 把 Prometheus 等收集到的数据变成漂亮的图表展示给你看。
- ELK/EFK: 主要解决日志的集中管理、查询、分析和告警问题。帮助你了解“发生了什么”。
- Feign: 使得在 Java 中调用其他 HTTP 服务变得更加简单，就像调用本地方法一样。



04 复制 微服务的应用模式

架构模式

- 微服务拆分
- 数据模式
- 测试模式
- UI模式
- 切面
- 安全
- 沟通模式
- 可观测性模式
- 部署模式



1-微服务架构模式

- 复杂的系统划分成微服务：

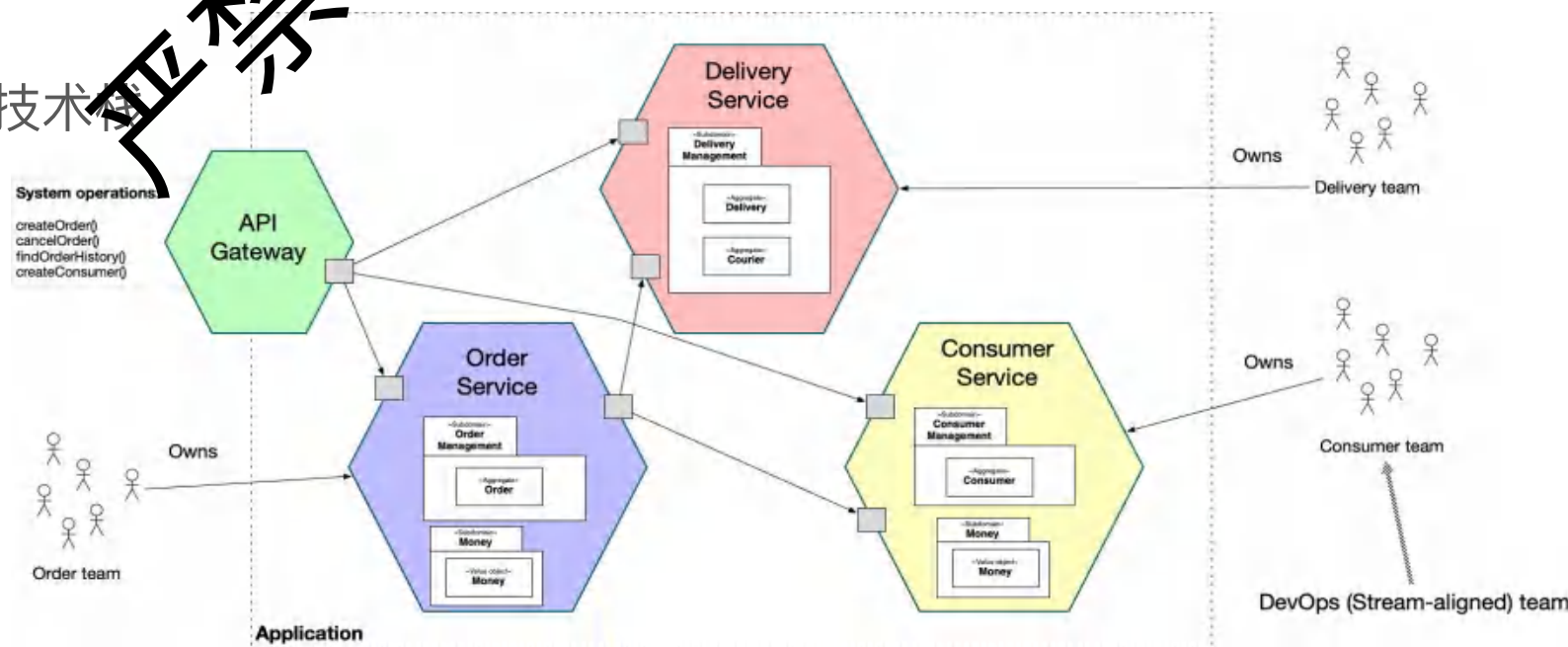
- 把系统结构化成2个或者多个独立部署，松耦合的组件或服务；
- 每个service包含1个或多个子领域；
- 每个子领域只属于一个service；
- 每个service归一个team负责；

- 优势

- 服务简单，团队自治，部署，技术栈

- 劣势

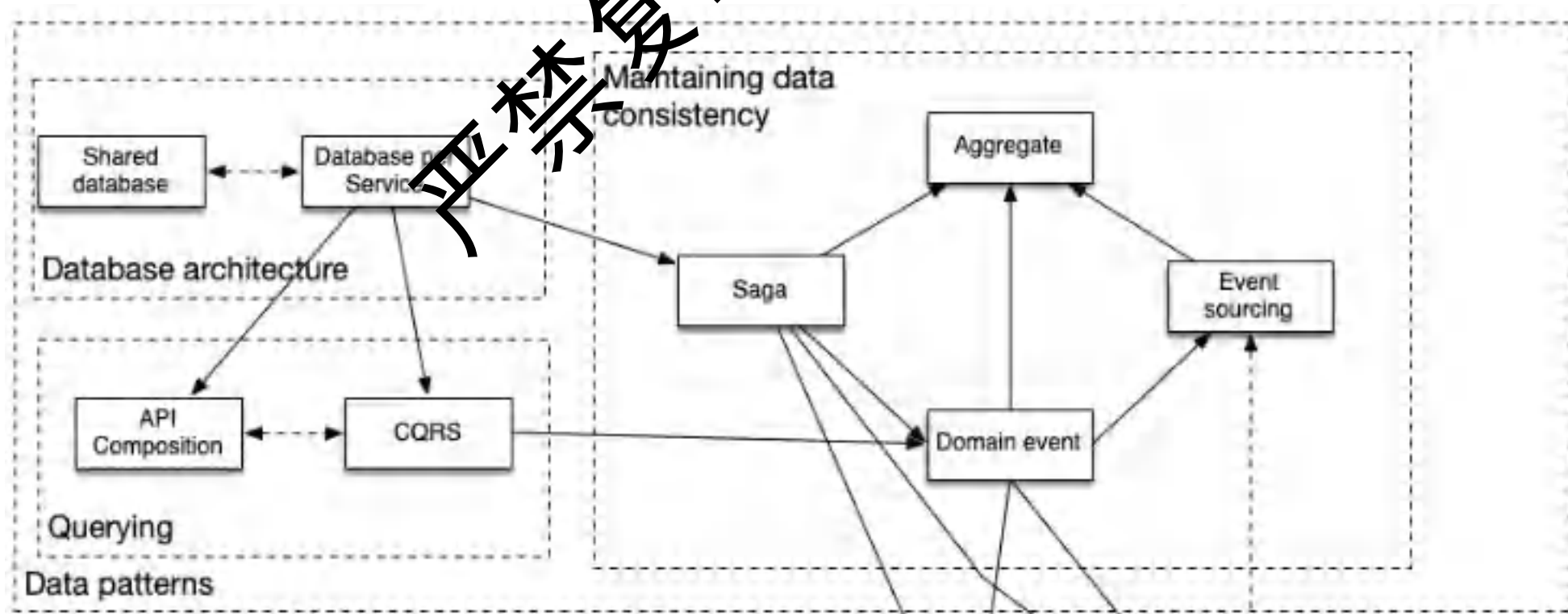
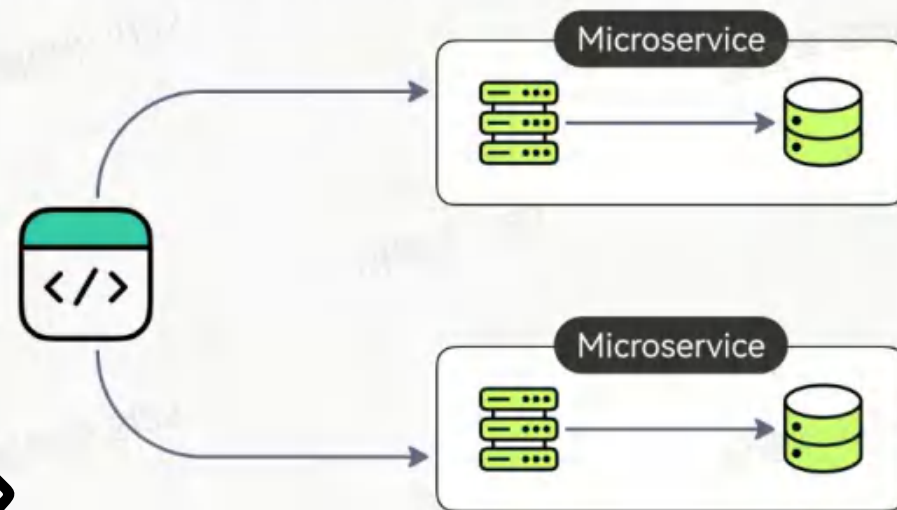
- 分布式事务管理



2-数据访问模式

- 数据库架构
 - 共享数据库
 - 每个服务都有自己的独立数据库
- 查询模式
 - API分解
 - CQRS
- 数据一致性
 - SAGA

Database Per Service Pattern



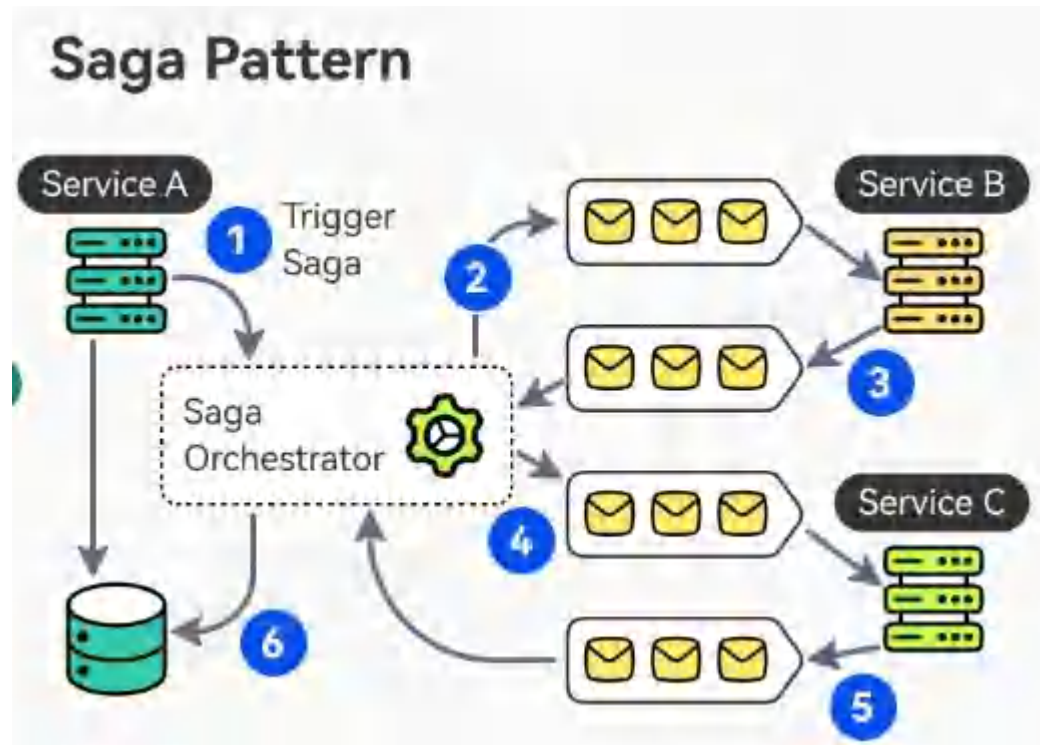
2-微服务数据模式-SAGA

- Saga模式的核心思想

- 将一个长事务拆分为一系列局部短事务
 - （称为“子事务”）；
- 每个子事务由不同的微服务负责；
- 每个子事务完成后会自动提交；
- 如果后续某个子事务失败，回滚之前的更改，从而保证业务的一致性：
 - Saga会通过执行前面已完成子事务的补偿操作（compensation action）

- 两种模式

- 编排（Orchestration）型 Saga
- 协作（Choreography）型 Saga



2 编排型saga模式

- 原理：
 - 发起方调用多个微服务完成工作；
 - 如果失败则回滚；
- 优势：
 - 代码比较简单
 - 同步调用，易于调试
- 劣势：
 - 耦合

严禁复制

2 编排型Saga

```
// OrderService.java
public class OrderService {

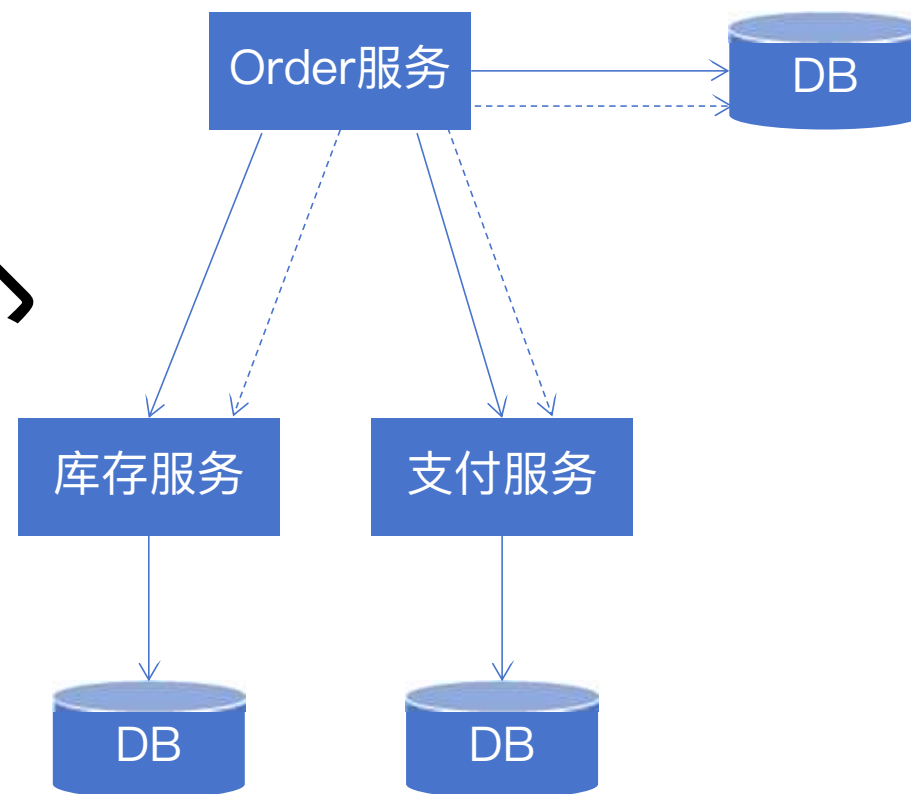
    private InventoryService inventoryService;
    private PaymentService paymentService;

    public void createOrder(Order order) {
        try {
            // 1. 创建订单 (本地事务)
            saveOrder(order);

            // 2. 扣减库存
            inventoryService.decreaseStock(order.getProductId(), order.getQuantity());

            // 3. 扣减余额
            paymentService.decreaseBalance(order.getUserId(), order.getAmount());

            // 4. 更新订单状态
            updateOrderStatus(order.getId(), "SUCCESS");
        } catch (Exception e) {
            // 补偿操作
            paymentService.refund(order.getUserId(), order.getAmount());
            inventoryService.increaseStock(order.getProductId(), order.getQuantity());
            updateOrderStatus(order.getId(), "FAILED");
            throw e;
        }
    }
}
```



2-协作型Saga模式

1. 订单服务：

1. 创建订单，
2. 发送OrderCreatedEvent。

2. 库存服务：

1. 监听，
2. 扣减库存，
3. 发送InventoryDeductedEvent。

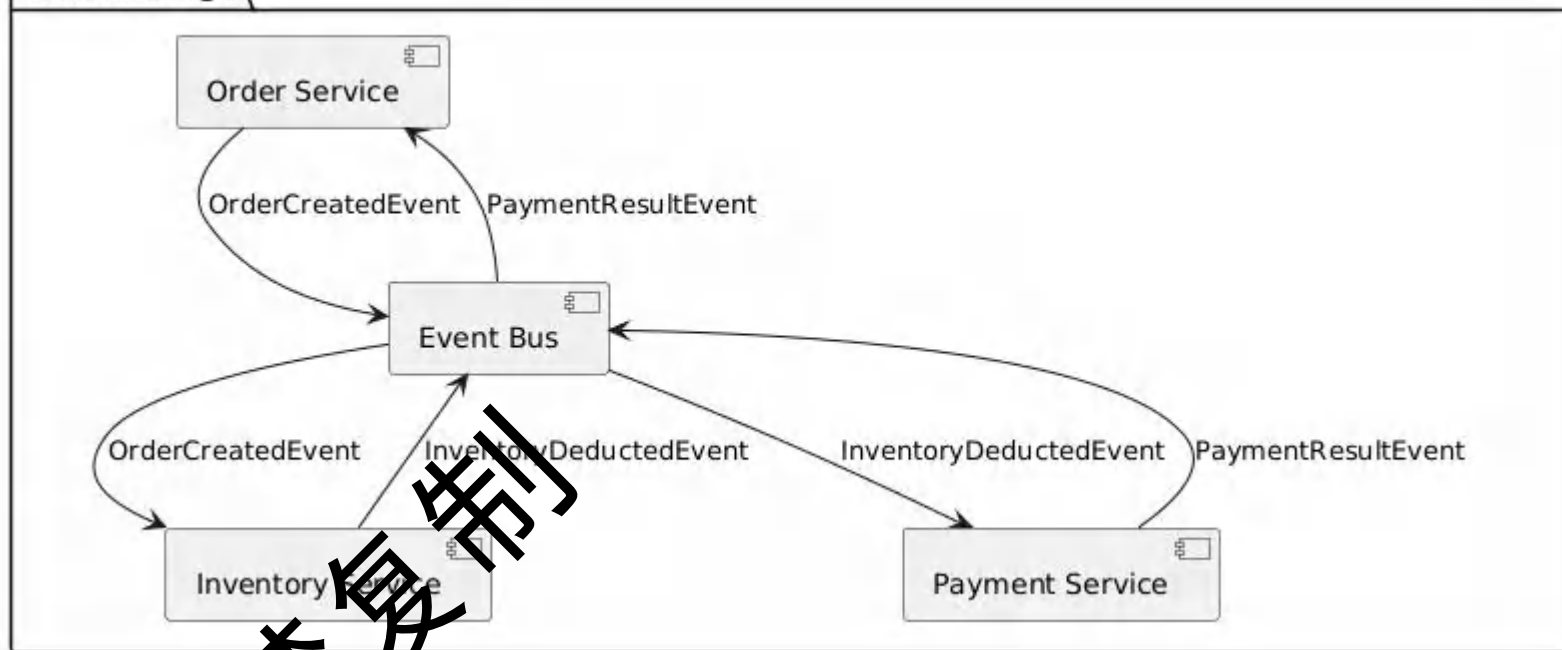
3. 支付服务：

1. 监听到库存成功事件，
2. 扣减余额，
3. 发送PaymentResultEvent。

4. 订单服务：

1. 监听支付结果，
2. 决定订单最终状态。

电商协作型Saga



```
@Component
public class InventoryService {
    @Autowired
    private ApplicationEventPublisher publisher;

    @EventListener
    public void handleOrderCreated(OrderCreatedEvent event) {
        boolean success = deductStock(event.getProductId(), event.getQuantity());
        publisher.publishEvent(new InventoryDeductedEvent(
            event.getOrderId(), success, success ? null : "库存不足"
        ));
    }
}
```

2-协作型Saga (Choreography)

优势

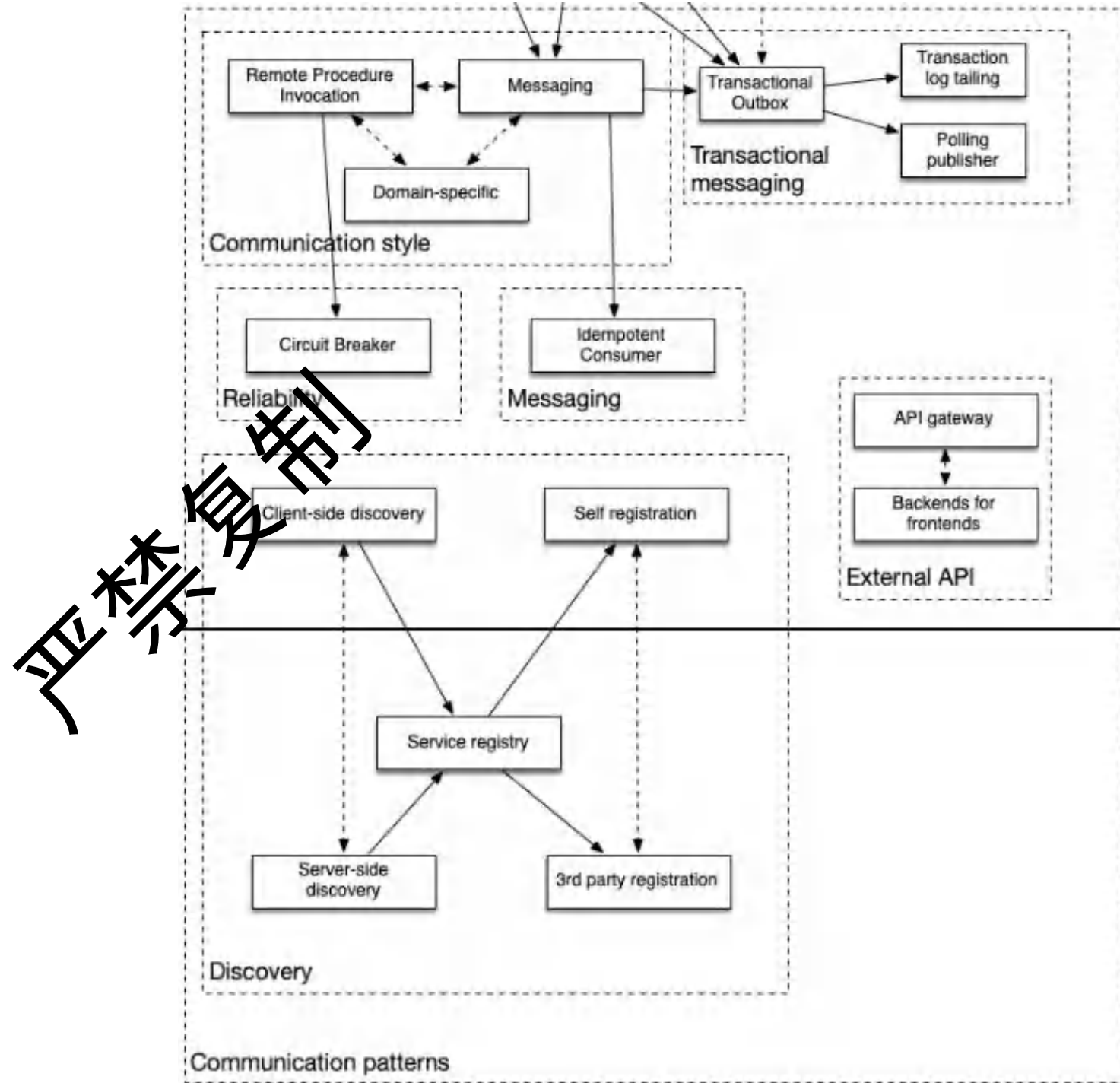
- 去中心化、无单点瓶颈
 - 无需专门的编排服务，弹性和可扩展。
- 高内聚、低耦合
 - 不直接调用其他服务，降低了服务耦合度。
- 易于扩展和维护
 - 新增服务只需订阅事件，利于系统演进。

劣势

- 业务流程可读性差，难以追踪
 - 整体流程不直观，排查问题和追踪流程困难。
- 补偿逻辑复杂
 - 多个服务协同补偿，补偿事件的顺序和幂等性。
- 事件风暴和顺序问题
 - 引起事件风暴；
 - 顺序和幂等性会引发一致性问题；
- 测试复杂
 - 端到端测试和集成测试难度较大。

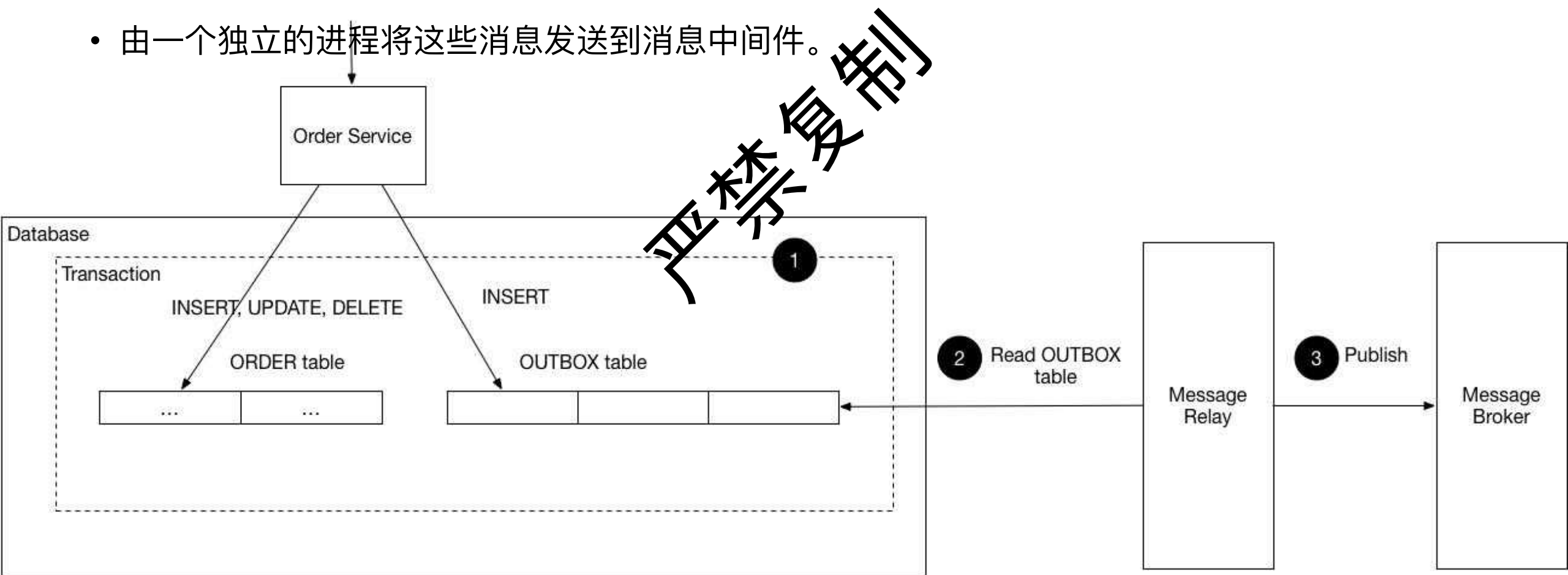
3-沟通模式

- 服务发现
 - 客户端发现，服务器端发现
 - 服务注册，自注册，第三方注册
- 沟通风格
 - RPC，消息，领域驱动
- 事务性消息
 - 事务性Outbox，
- 可靠性
 - 熔断保护
- 消息
 - 幂等消费



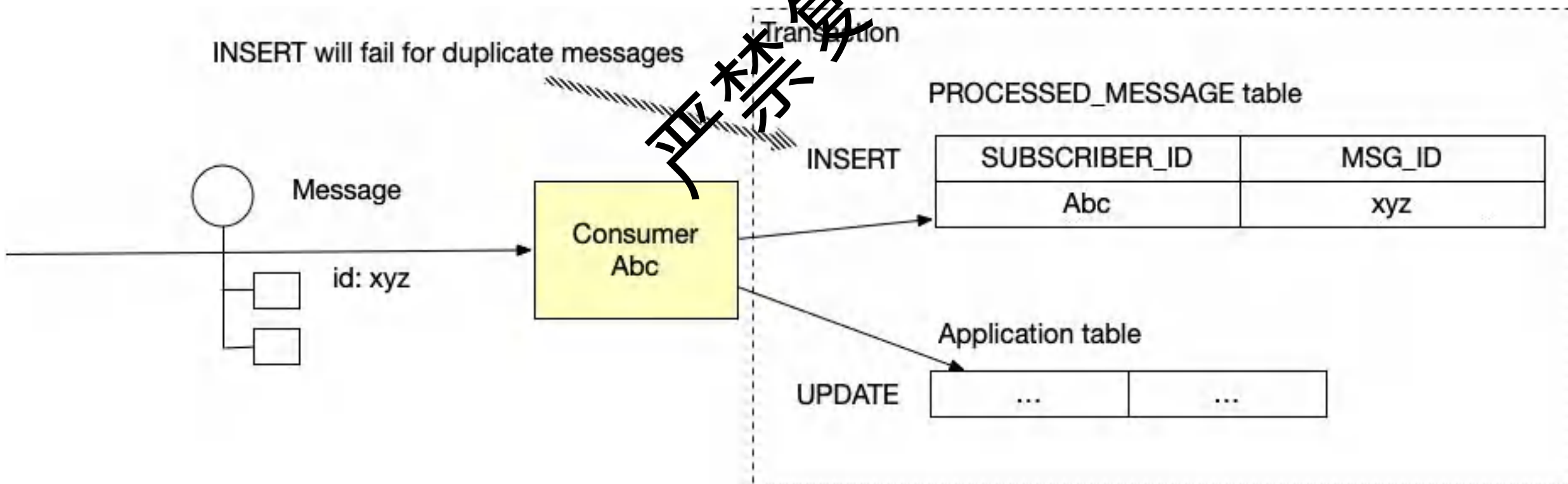
4-事务性outbox模式

- 如何才能保证修改数据库和发送消息在同一个事务内完成？
- 发送消息的服务：首先insert order+insert msg。
- 由一个独立的进程将这些消息发送到消息中间件。



4-幂等消费者模式

- 消息队列可能把同一个消息多次发给consumer，如何保证正确性？
- consumer把接受到的消息存储到table中，如果接受到消息insert失败则不处理。



5 servicemesh sidecar

- 目标:

- 流量控制:

- 所有服务间的调用都经过 Sidecar 代理, Service Mesh 可以实现流量监控、限流、重试等。

- 安全通信:

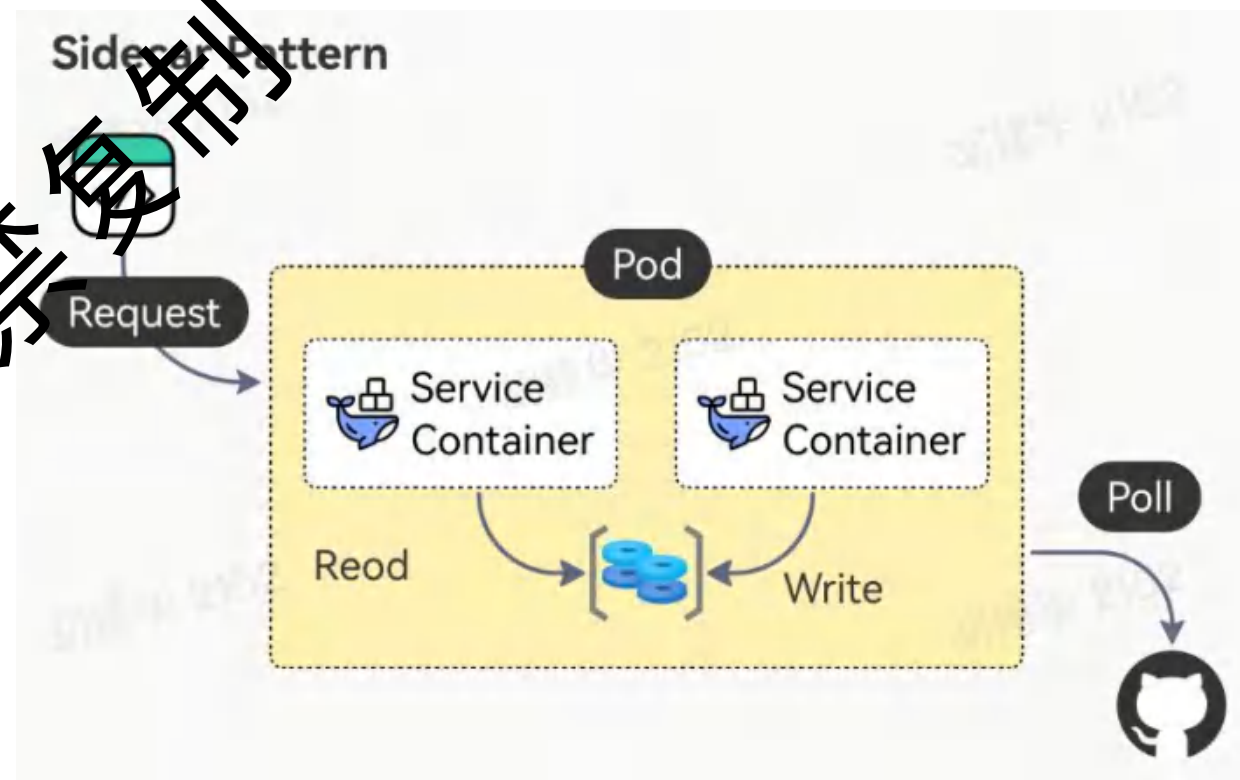
- 自动为服务间通信加密, 无需开发者关心

- 可观测性:

- 自动收集调用链路、日志和指标, 便于监控和排查问题。

- 无侵入:

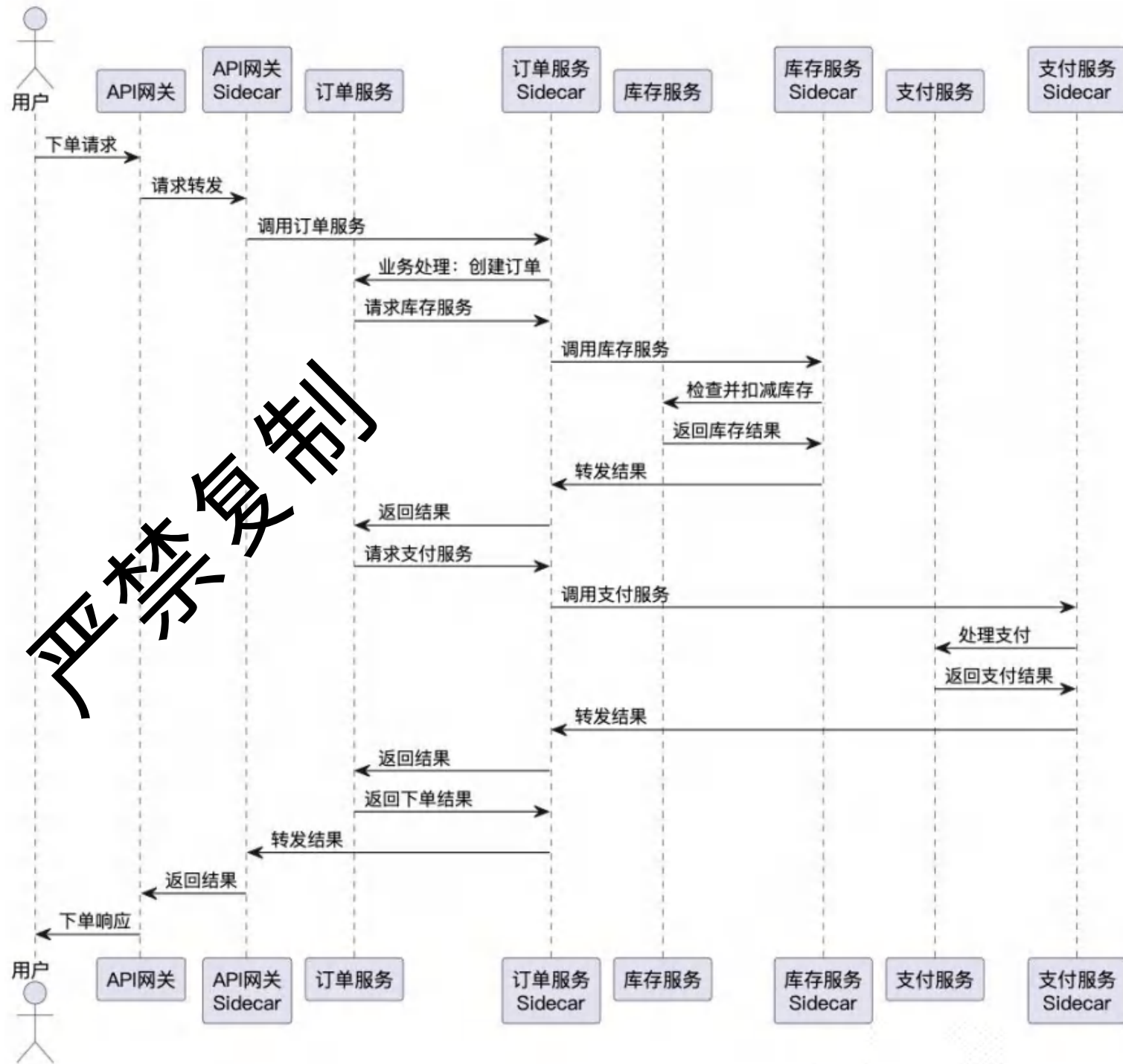
- 业务代码无需关注网络通信细节, 全部由 Service Mesh 代理透明处理



5 Servicemesh例子

1. API网关Sidecar 调用 订单服务的Sidecar;
2. 订单服务的Sidecar再调用订单服务;

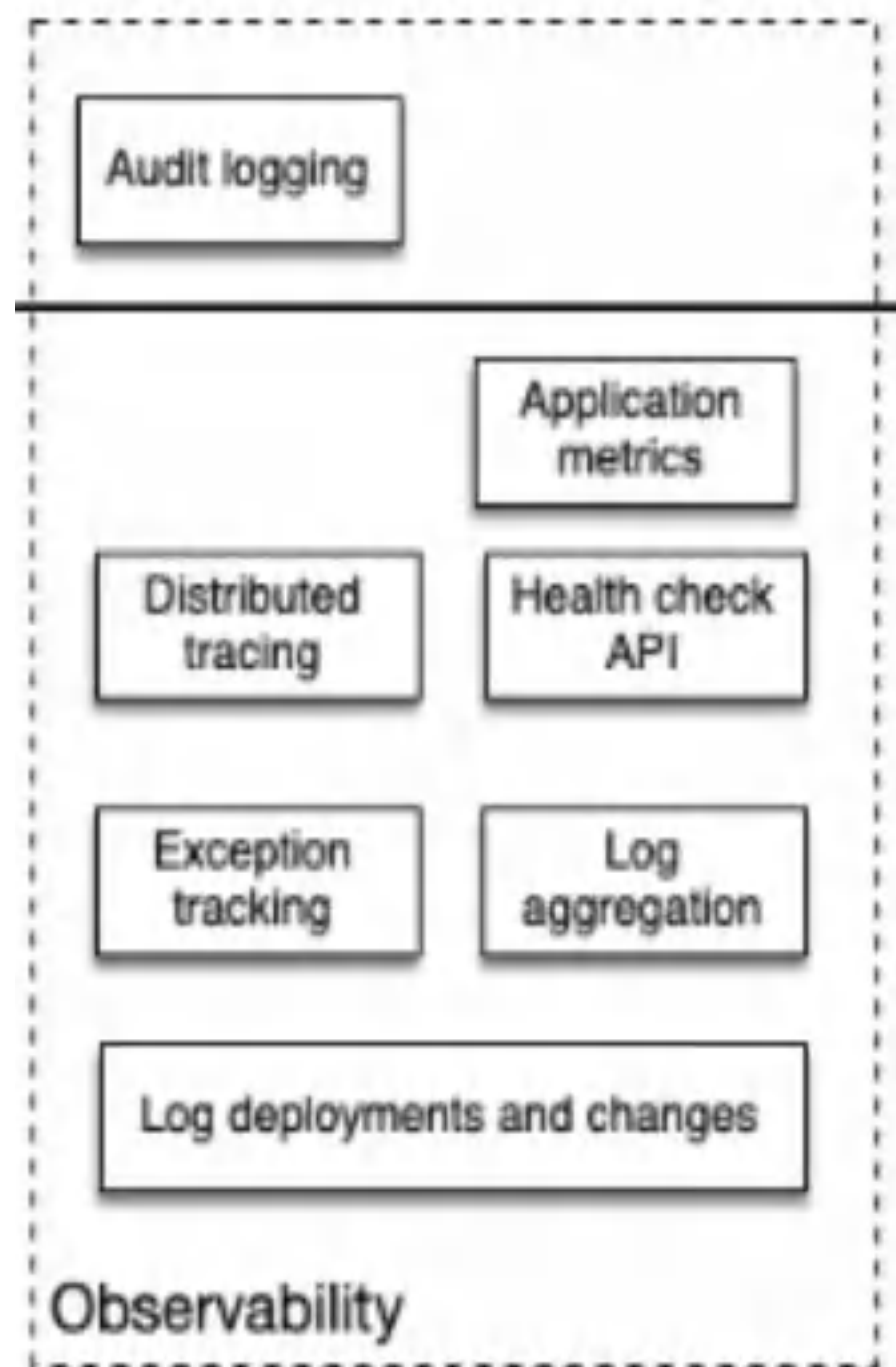
比喻：sidecar是服务的帽子。



6 可观察性模式

- 日志聚合
- 应用系统指标监控
 - 请求数，响应时间
- 审核日志
- 分布式tracing
- 异常日志跟踪
- 健康检查API

严禁复制



6 可观察性模式

<https://www.elastic.co>

Services

Environment

All



Search transactions, errors and metrics (E.g. transaction.duration.us > 300000 AND http.response.status_code >= 400)



Last 30 minutes

Show dates

Refresh

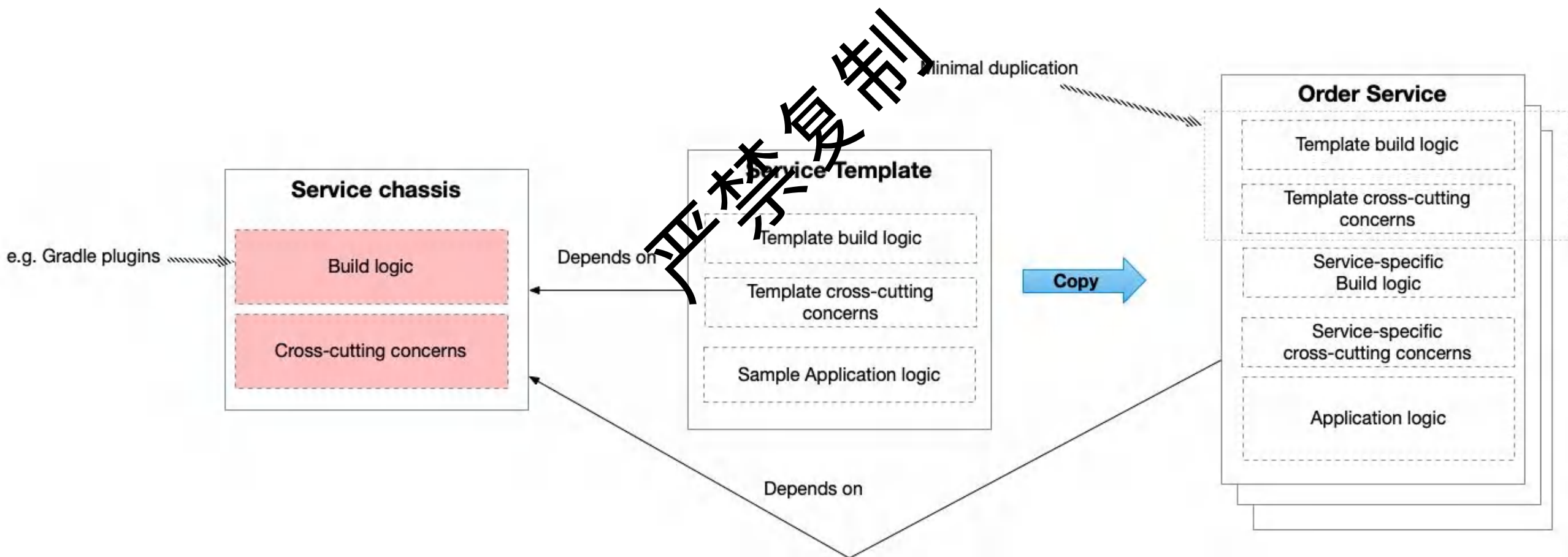
? What are these metrics?

Health ↓	Name	Environment	Latency (avg.)	Throughput	Error rate %
Healthy	opbeans-node	production	117 ms	145.5 tpm	4.9%
Healthy	apm-server		2,950 ms	63.8 tpm	0%
Healthy	opbeans-go	production	288 ms	27.0 tpm	2.2%
Healthy	opbeans-java	production	236 ms	26.8 tpm	4.9%
Healthy	opbeans-dotnet	production	671 ms	26.2 tpm	0%
Healthy	JS client		428 ms	2.8 tpm	N/A

7 微服务底座模式

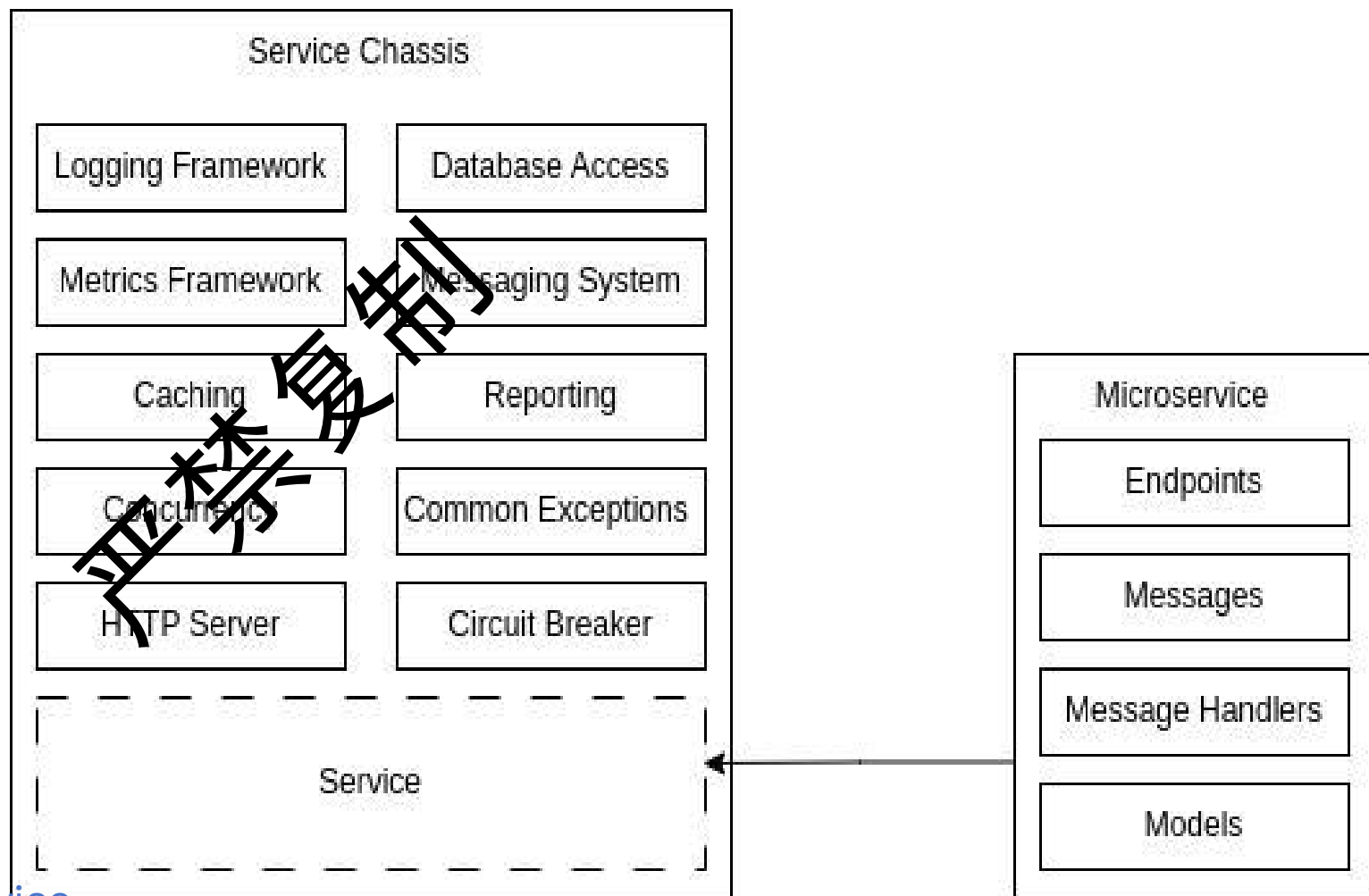
<https://microservices.io/patterns/microservice-chassis.html>

- 问题：研发团队如何才能快速的创建一个微服务？
- 创建一个微服务底座，使用微服务template，其它微服务基于它们进行开发。



7 微服务底座模式

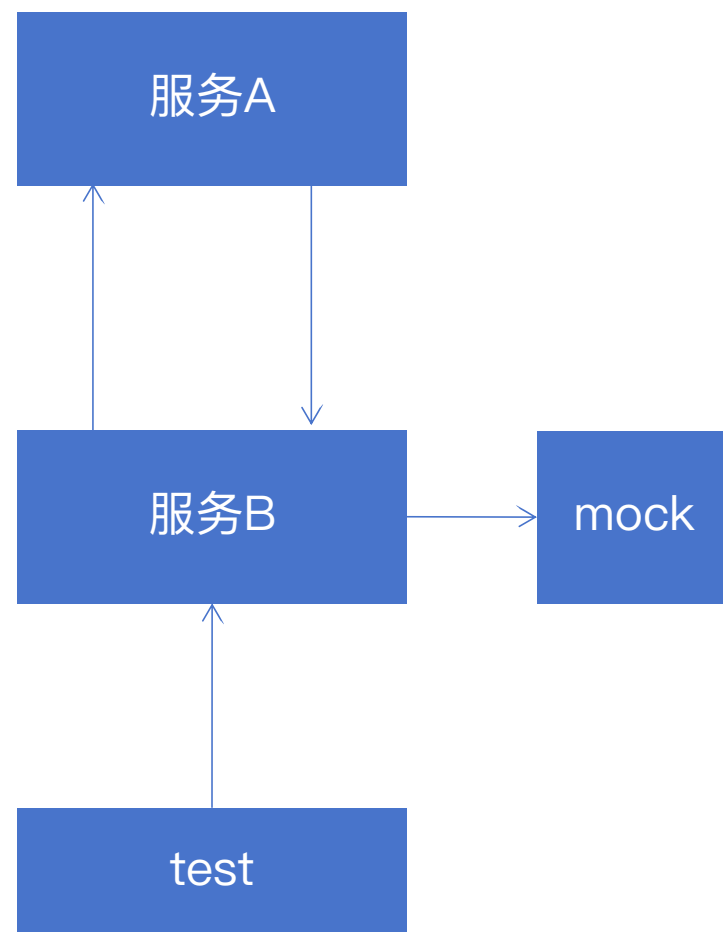
- 标准化；
- 更快的开发；
- 安全增强；
- 简化运维；
- 简化集成和测试；
- 用户选择不同的技术开发



<https://radicalgeek.co.uk/microservice-architecture/building-microservices-the-service-chassis/>

8 微服务Testing模式

- 目标
 - 自动化验证service的正确性
- 解决方案
 - 消费驱动的接口测试：验证service是否正确的提供了接口功能
 - 消费端接口测试：验证消费端是否正确的调用接口
 - 服务组件测试：测试服务自身的逻辑，mock外部服务



9 微服务UI模式

- 模式1： 服务端页面片段组装
 - 在服务端构建最终网页，将多个业务能力/子域相关的Web应用生成的HTML片段拼接成完整页面。
 - 实现方式：
 - 用户请求页面时，网关或页面聚合服务会向各个后端微服务请求各自负责的HTML片段。
 - 每个微服务返回对应的HTML片段（如订单信息、用户信息、推荐列表等）。
 - 聚合服务将这些片段组装成一个完整的HTML页面，返回给前端浏览器。
- 模式2： 客户端UI组装
 - 在客户端（如浏览器）组装页面，将多个子域相关的UI组件动态组合，各自的微服务获取数据并渲染。
 - 实现方式
 - 如React、Vue、Web Components等。
 - 各子组件独立向自己的后端服务请求数据（通常是REST API或GraphQL）。
 - 组件渲染各自的数据，最终在页面上组合成完整UI。
 - 适用于：
 - 需要高度动态交互的SPA（单页应用）、管理后台、仪表盘等

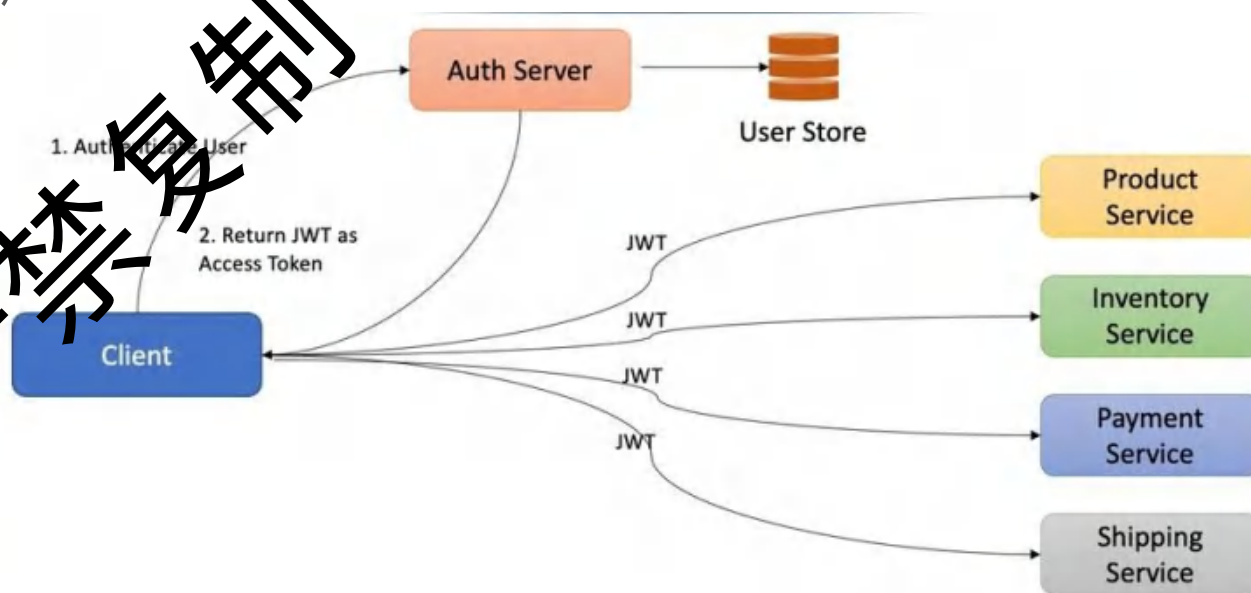
10 安全性 JWT

- 定义：

- 一种基于JSON的轻量级认证和授权令牌格式，用于在各系统之间安全地传递用户身份信息。

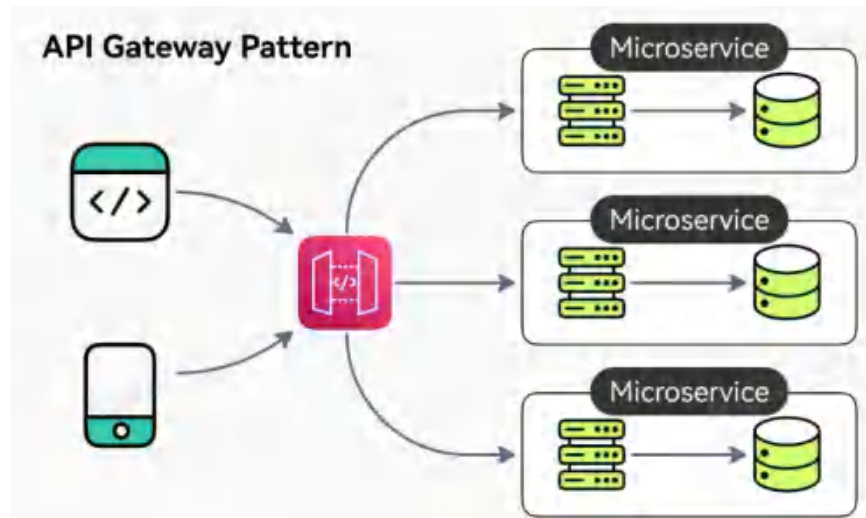
- 内容：

- Header（头部）：
 - 声明类型（typ）和签名算法（alg）
- Payload（负载）：
 - 如用户ID、过期时间、权限等
- Signature（签名）：
 - 对前两部分进行加密签名，防止数据被篡改。



11 API Gateway模式

- 定义：对微服务的访问都通过gateway进行
- 单一入口点 (Single Entry Point), 位于客户端和后端微服务之间
 - 复杂性：寻址，协议不同
 - 耦合性：客户端和后端服务耦合
 - 网络开销：多次往返调用
 - 横切面关注：认证、授权、限流、日志、监控、SSL 终止



```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
      predicates:
        - Path=/order/**
```

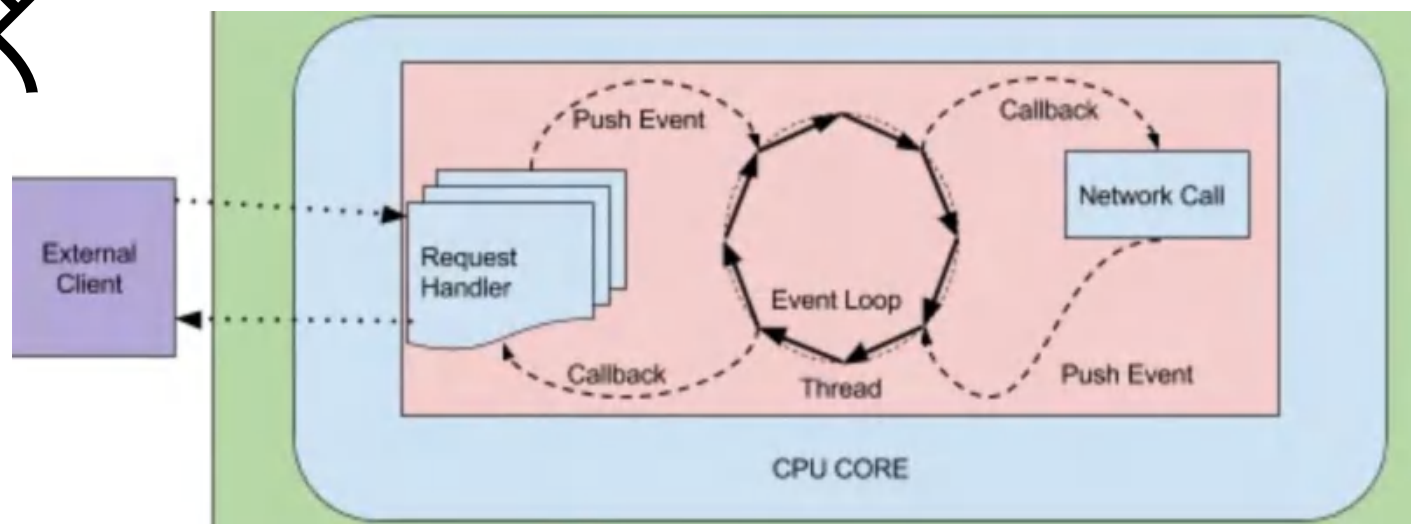
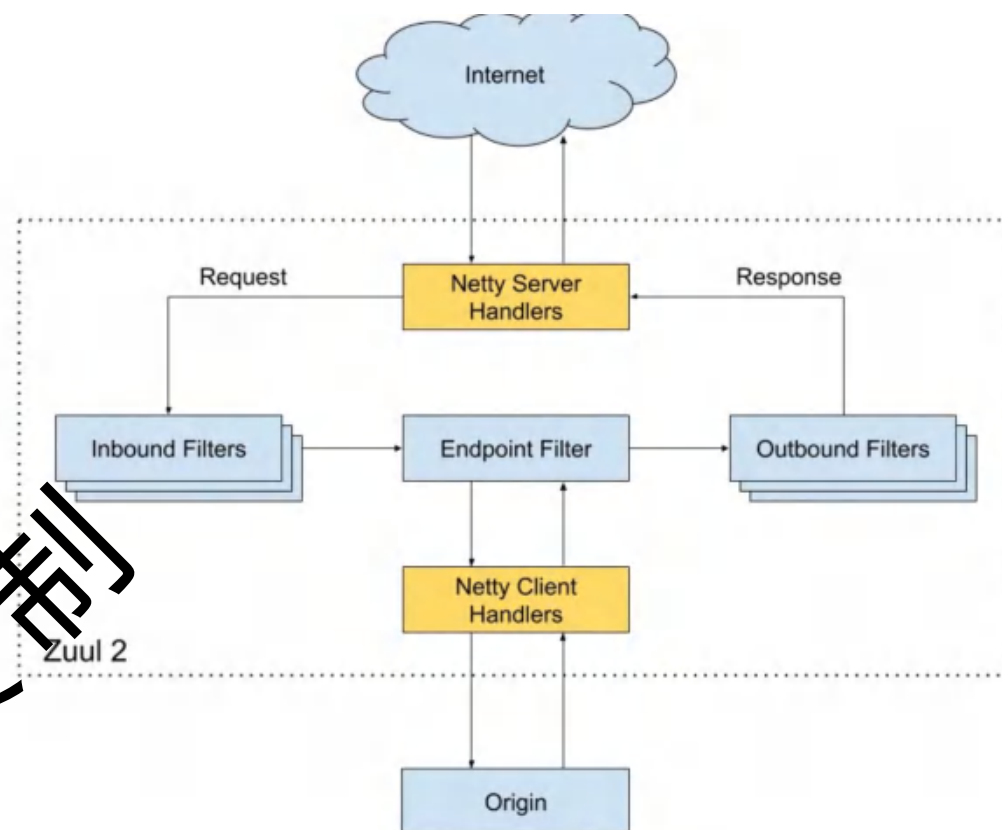
```
String gatewayUrl = "http://localhost:8080/order/info";
String result = restTemplate.getForObject(gatewayUrl, String.class);
```

12 API Gateway: Zuul

- 基于 Netty 的异步非阻塞架构
- 高性能和高吞吐量
- 弹性与容错能力
- 路由和负载均衡
- 支持多种协议
- 动态配置和热加载
- 可观测性

<https://netflixtechblog.com/zuul-2-the-netflix-journey-to-asynchronous-non-blocking-systems-45947377fb5c>

严禁复制



THANK YOU

北京交通大学软件学院

严禁复制

